DOCUMENTO DE EVIDENCIAS

Base de datos del cine

Creación de tablas, inserción de datos, triggers, procedimientos y consultas SQL

Grupo 4

Motor: PostgreSQL en Neon (base de datos neondb)

1. Creación de tablas

Se crearon las ocho tablas del modelo con sus llaves primarias, llaves foráneas y restricciones. En cada captura, Neon muestra “Statement executed successfully”, lo que confirma que la tabla se creó sin errores.

Salas

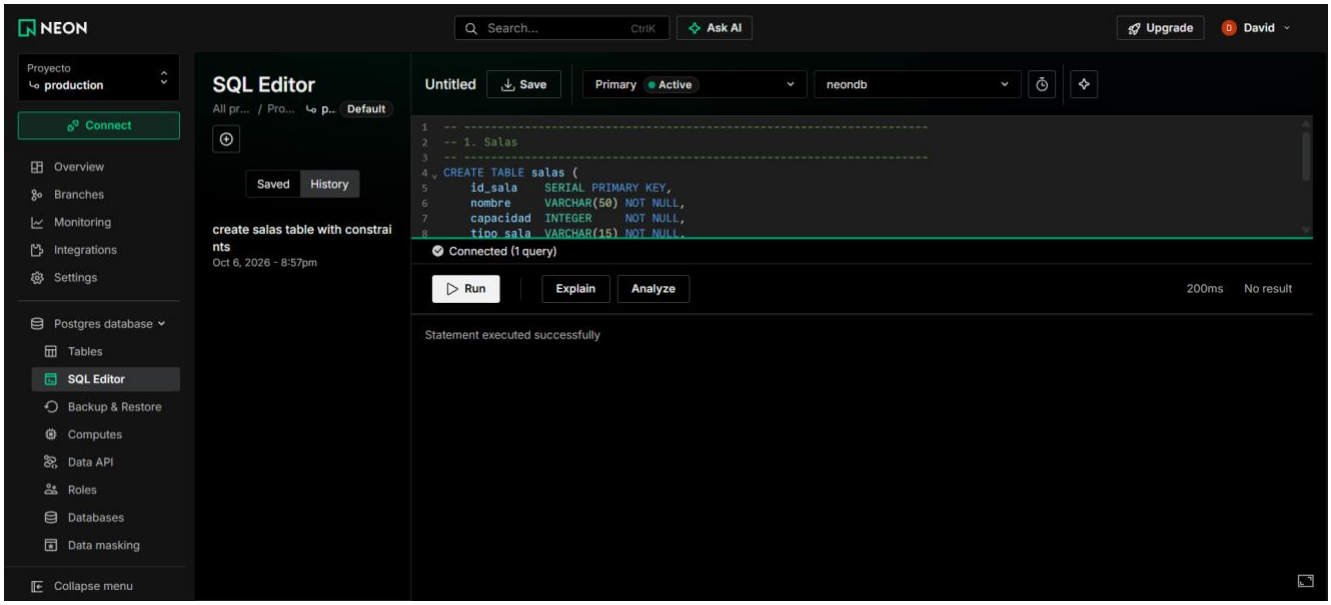


Figura 1. *Creación de la tabla salas en el SQL Editor de Neon.*

Actores y directores

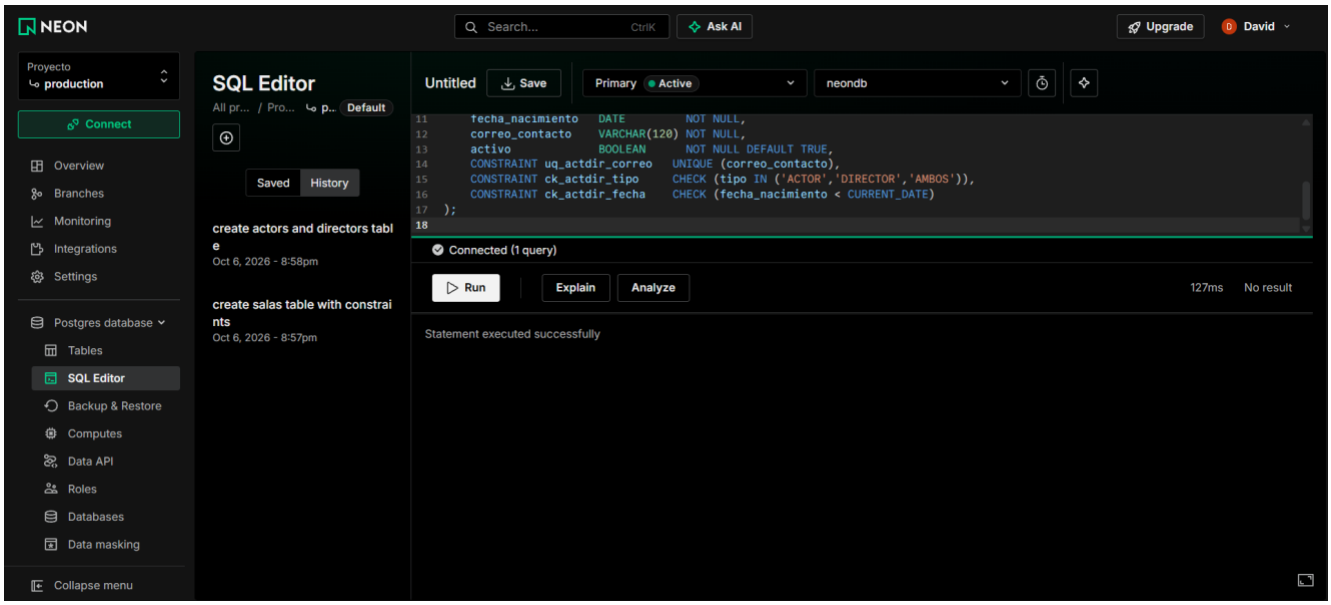


Figura 2. *Creación de la tabla actores y directores en el SQL Editor de Neon.*

Tipos de boleta

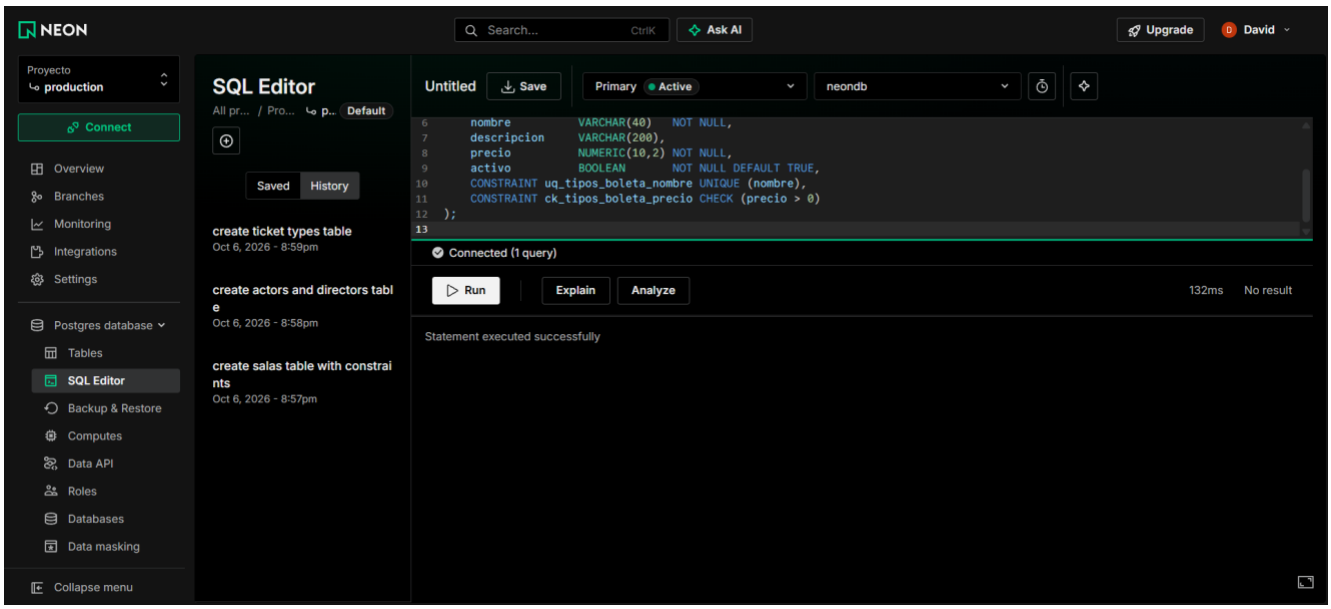


Figura 3. *Creación de la tabla tipos de boleta en el SQL Editor de Neon.*

Espectadores

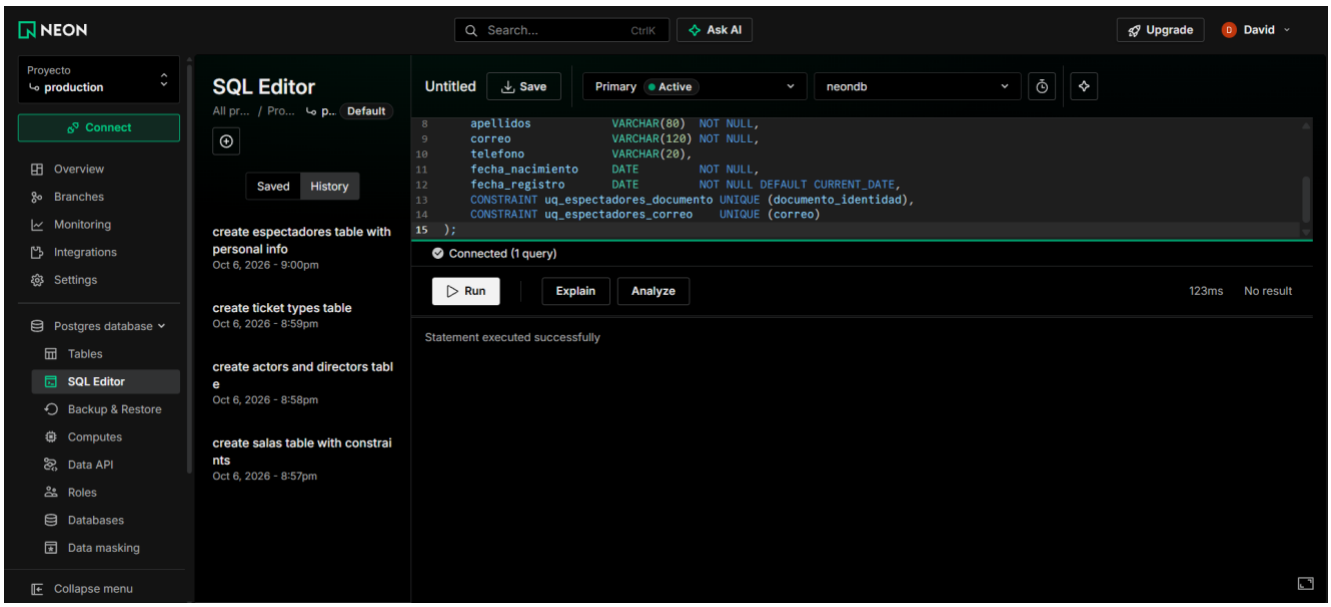


Figura 4. *Creación de la tabla espectadores en el SQL Editor de Neon.*

Staff

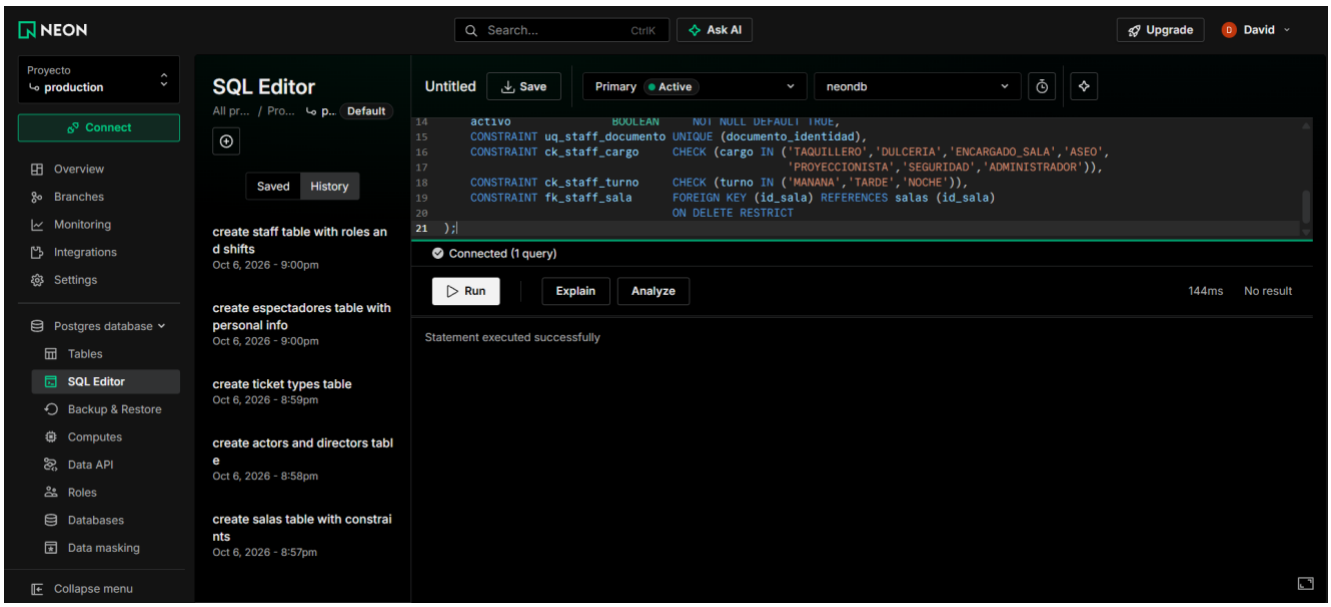


Figura 5. *Creación de la tabla staff en el SQL Editor de Neon.*

Contratos

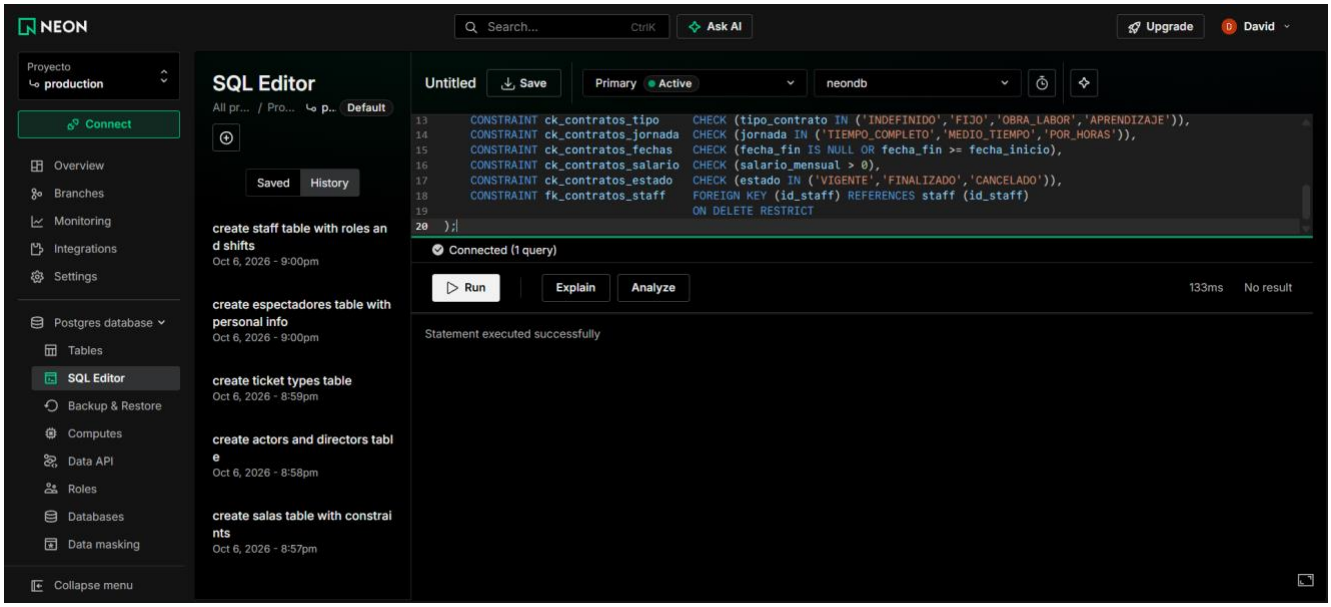


Figura 6. *Creación de la tabla contratos en el SQL Editor de Neon.*

Funciones

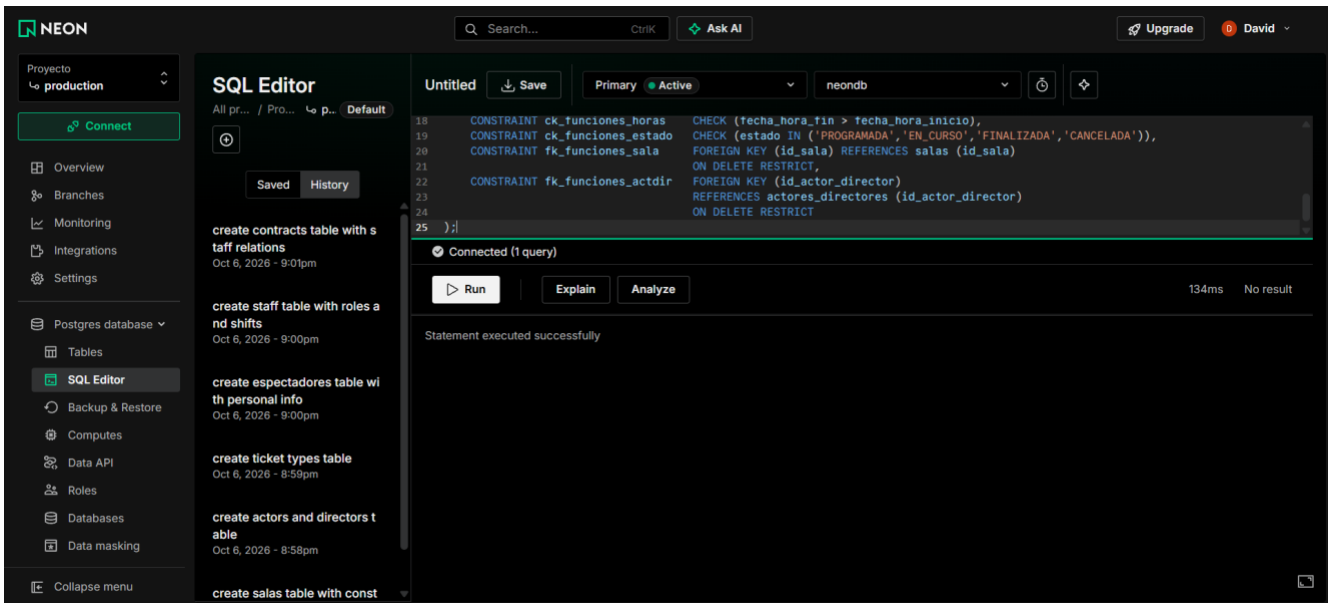


Figura 7. *Creación de la tabla funciones en el SQL Editor de Neon.*

Ventas

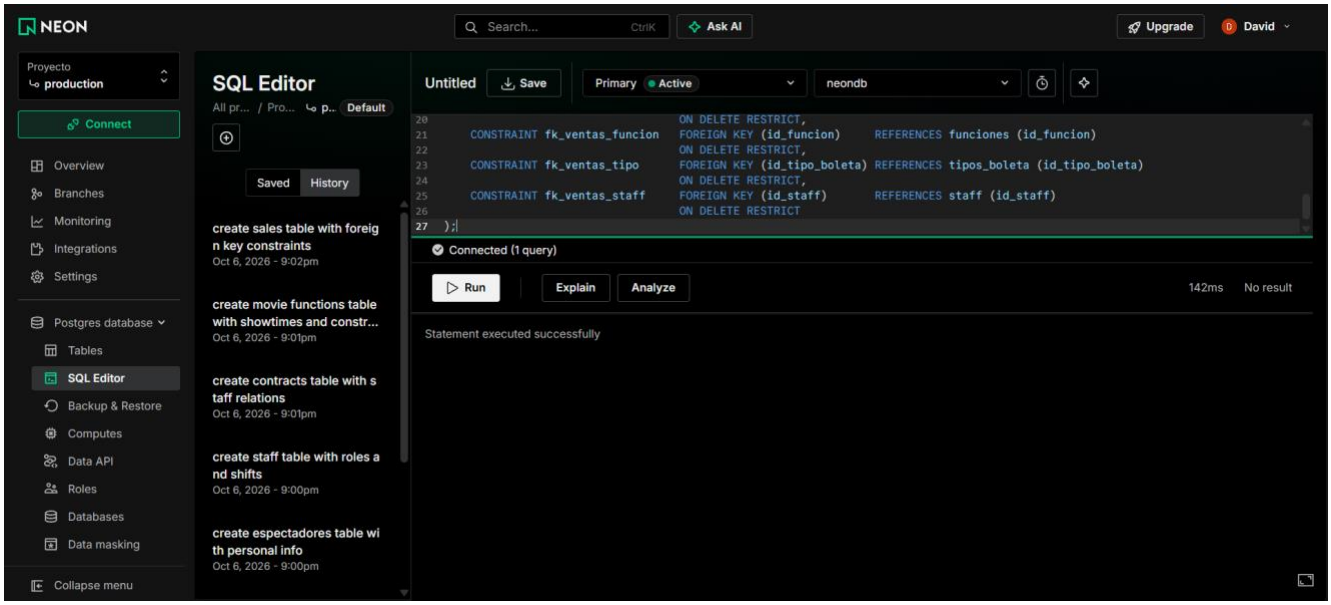


Figura 8. *Creación de la tabla ventas en el SQL Editor de Neon.*

2. Inserción de datos

2.1 Limpieza e inserción

BEGIN abre una transacción, es decir, todo lo que viene después se aplica todo o no se aplica nada, y el COMMIT del final lo confirma. El TRUNCATE TABLE vacía las ocho tablas en una sola vez, y como ese comando lo hago con RESTART IDENTITY, los contadores SERIAL empiezan otra vez desde el 1, algo necesario porque los inserts usan IDs fijos como llaves foráneas (por ejemplo id_sala = 3) y se desajustarían si los contadores siguieran donde quedaron. El CASCADE hace que se vacíen también las tablas que dependan de las listadas, aunque aquí ya van todas incluidas, así que es sólo una garantía extra. Esto, en conjunto, permite ejecutar el script las veces que uno quiera y obtener siempre exactamente los mismos datos.

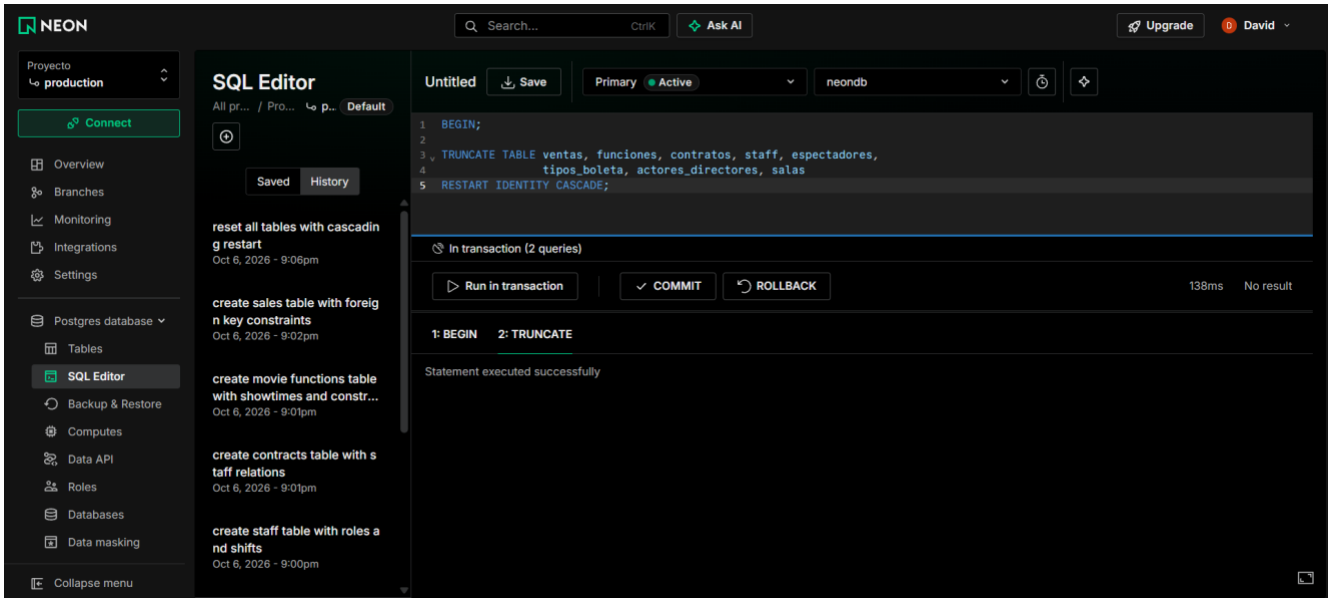


Figura 9. *Apertura de la transacción (BEGIN) y vaciado de las ocho tablas con TRUNCATE ... RESTART IDENTITY CASCADE.*

A continuación se insertaron todos los datos dentro de la misma transacción y se confirmaron con COMMIT. Neon muestra una pestaña por cada INSERT (5, 5, 32, 32, 30, 30, 32 y 45 filas) y la pestaña final COMMIT.

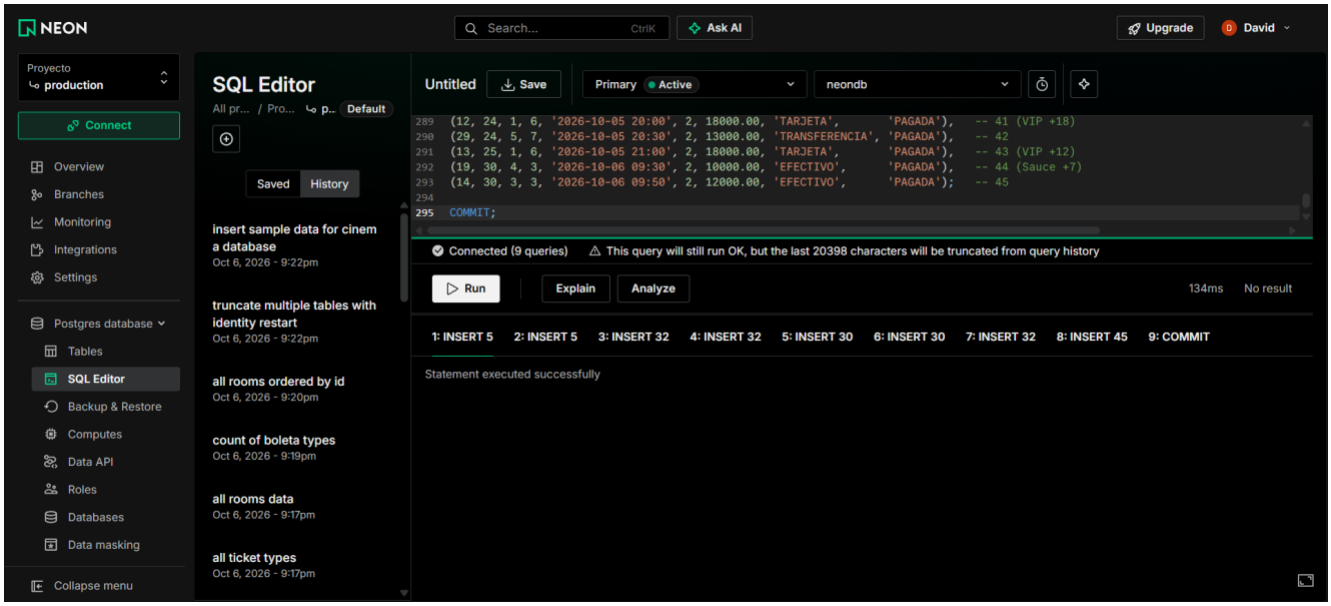


Figura 10. *Inserts de las ocho tablas y COMMIT final; todas las sentencias se ejecutaron con éxito.*

2.2 Conteo de registros

Para confirmar que la carga quedó completa se contó el número de registros de cada tabla.

Tabla	Registros
salas	5
tipos_boleta	5
actores_directores	32
espectadores	32
staff	30
contratos	30
funciones	32
ventas	45

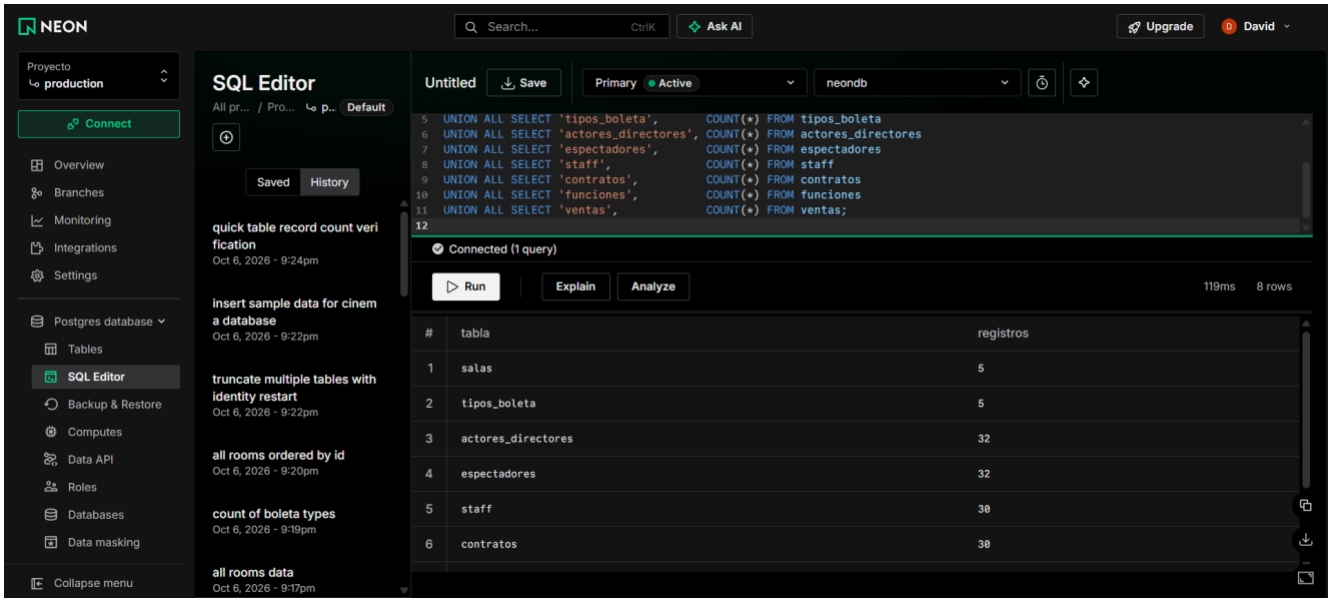


Figura 11. *Resultado del conteo de registros por tabla (filas 1 a 6).*

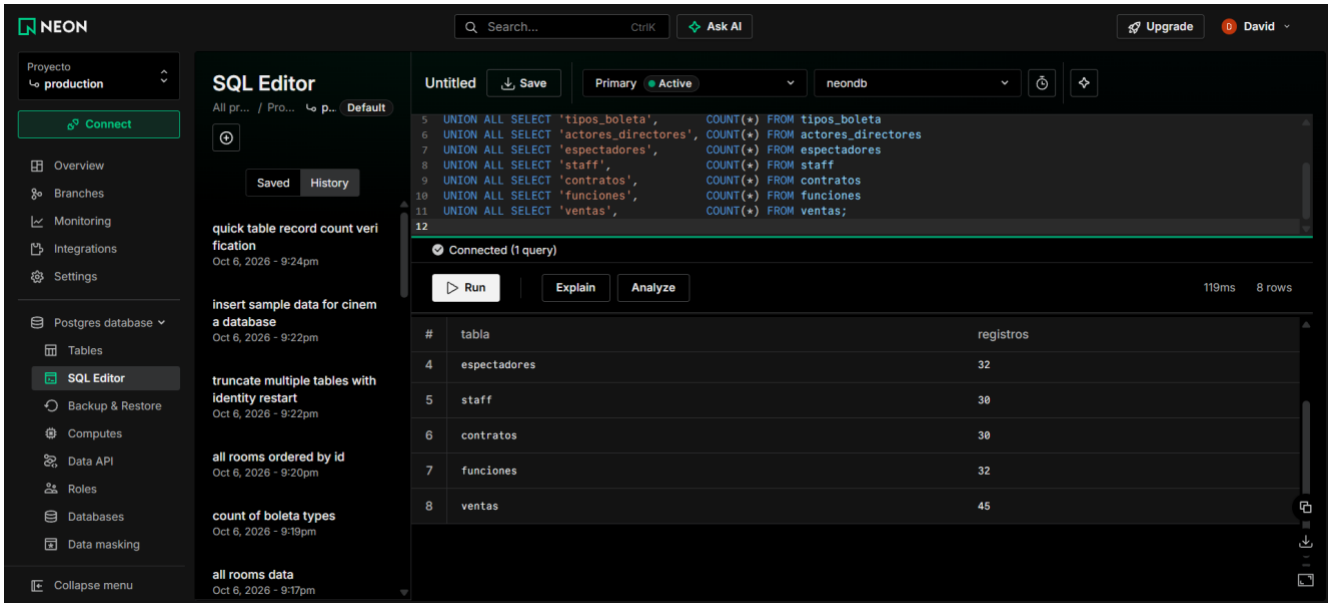


Figura 12. *Resultado del conteo de registros por tabla (filas 4 a 8).*

2.3 Verificación rápida de cada tabla

Se consultó el contenido de cada tabla con SELECT * para comprobar que los datos quedaron registrados correctamente.

Salas

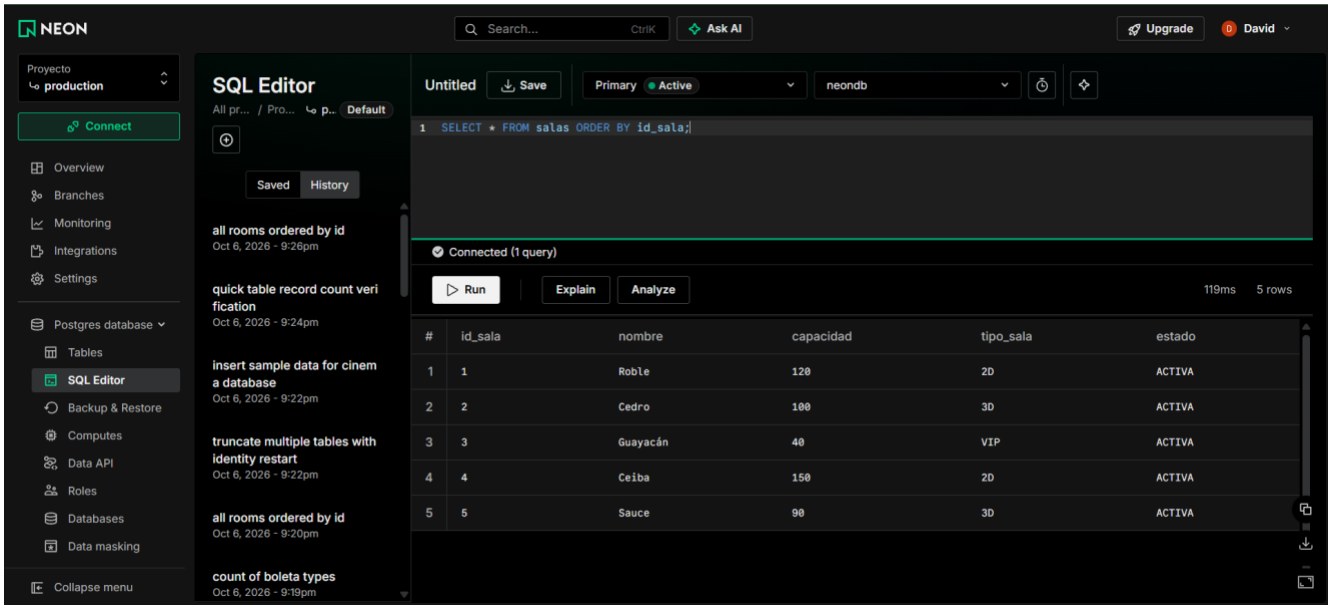


Figura 13. *Contenido de la tabla salas.*

Tipos de boleta

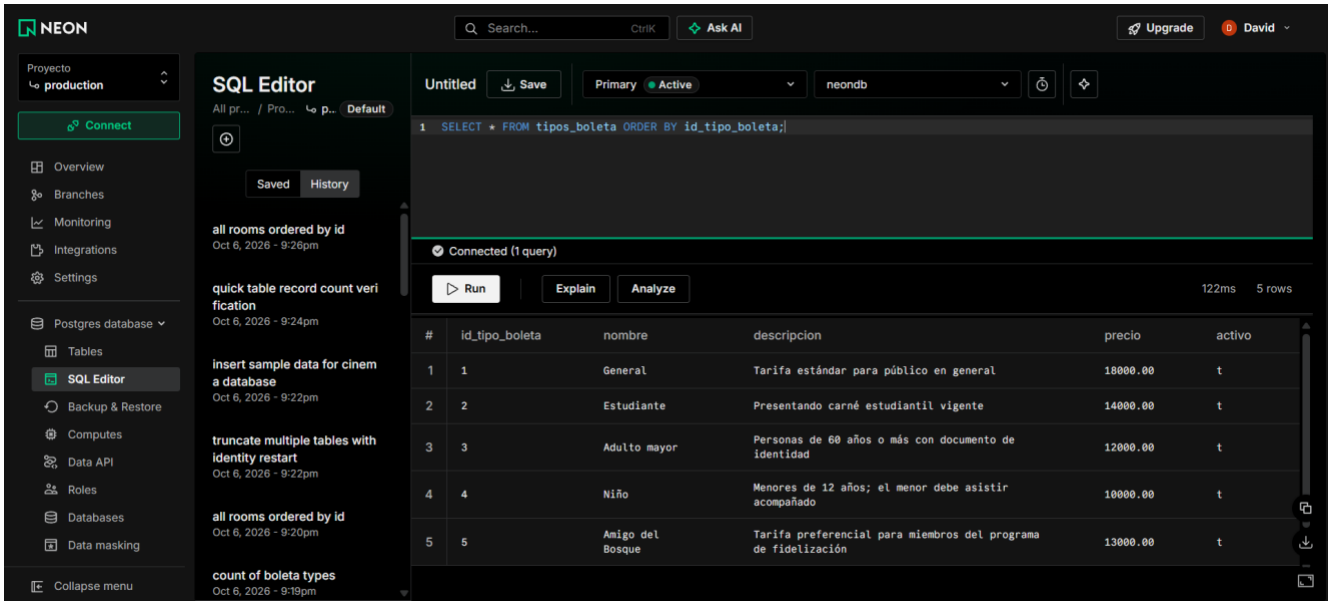


Figura 14. *Contenido de la tabla tipos de boleta.*

Actores y directores

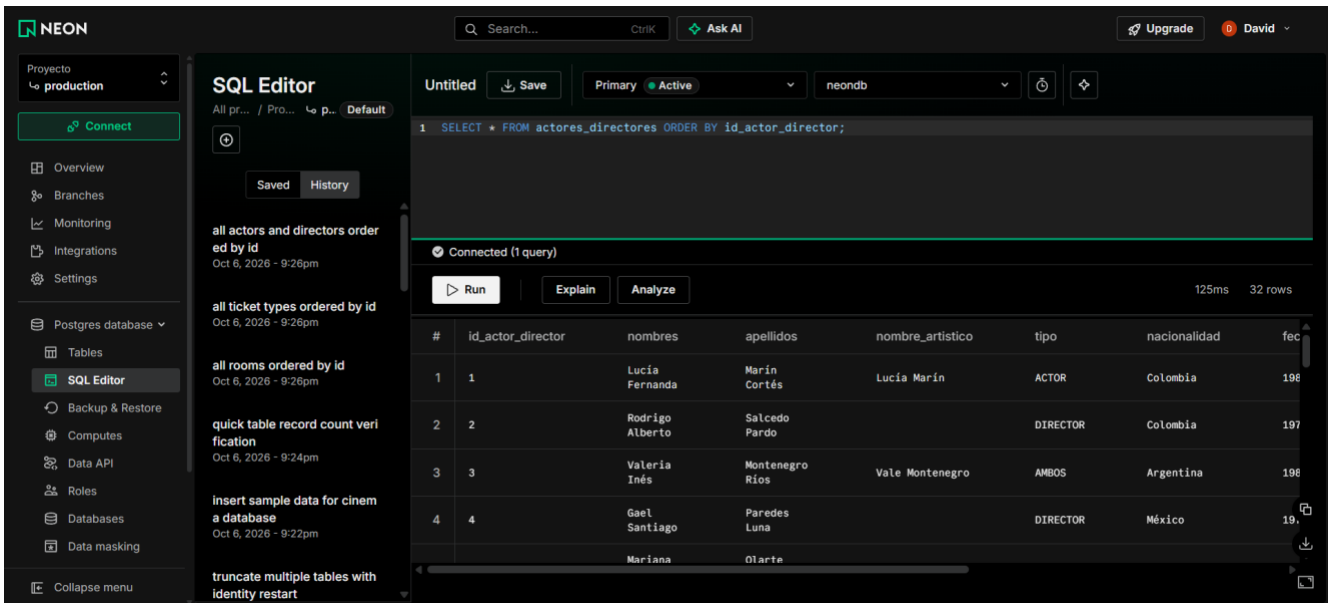


Figura 15. *Contenido de la tabla actores y directores (captura 1 de 2).*

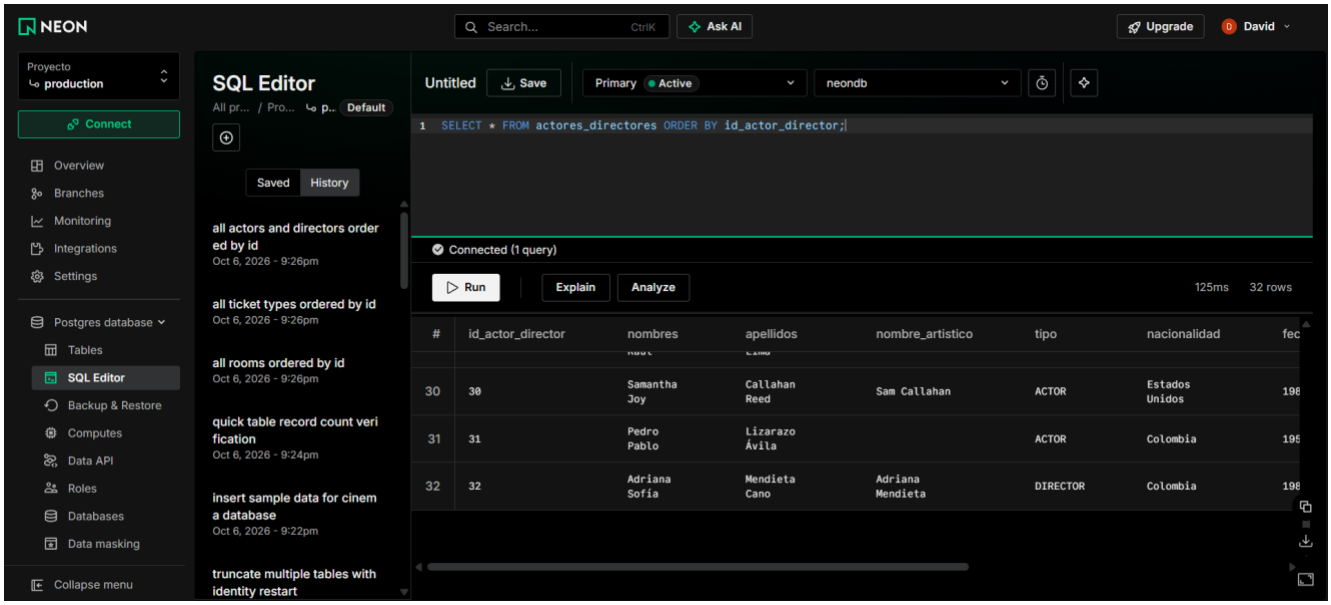


Figura 16. *Contenido de la tabla actores y directores (captura 2 de 2).*

Espectadores

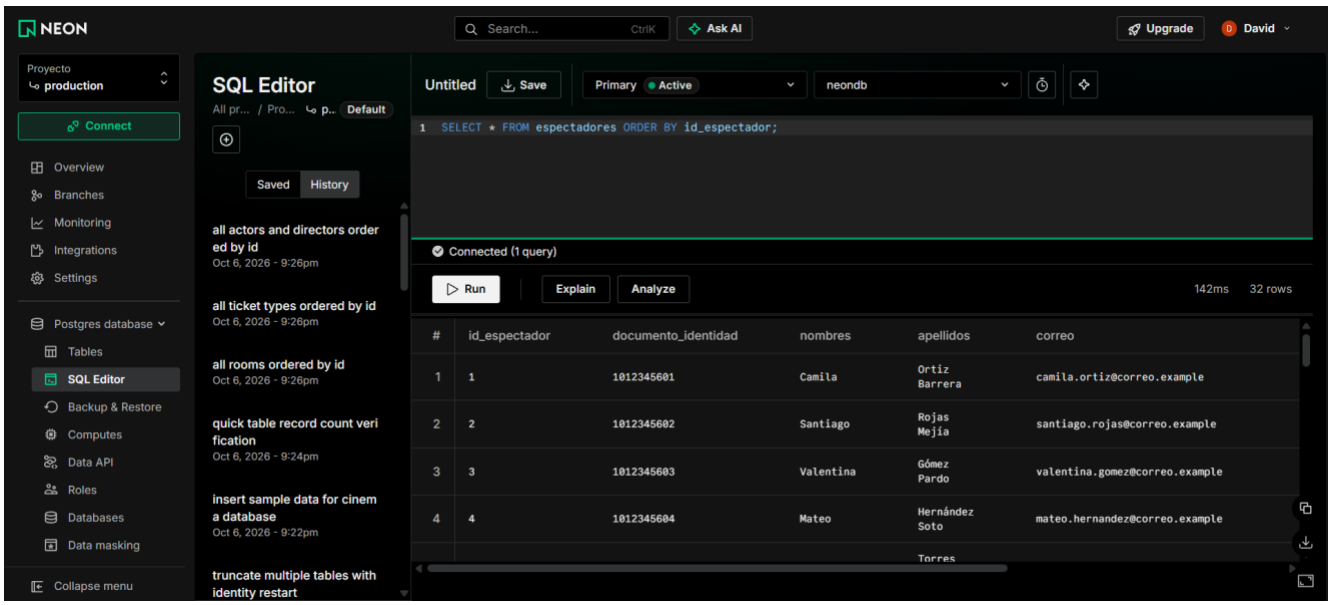


Figura 17. *Contenido de la tabla espectadores (captura 1 de 2).*

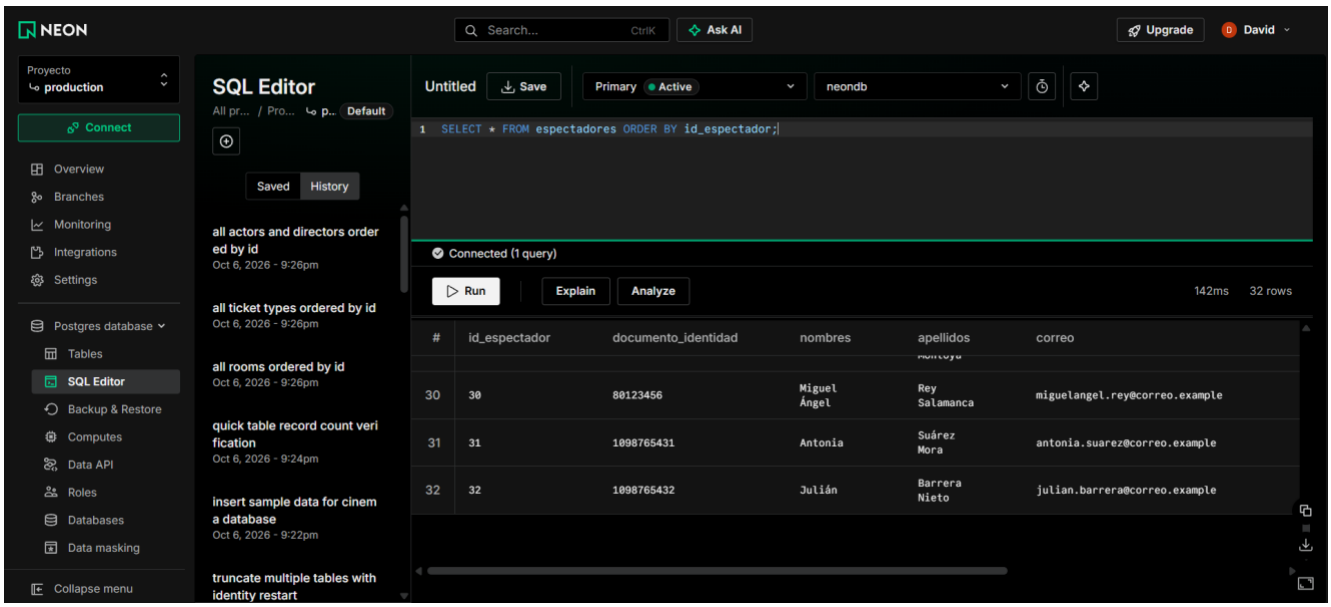


Figura 18. *Contenido de la tabla espectadores (captura 2 de 2).*

Staff

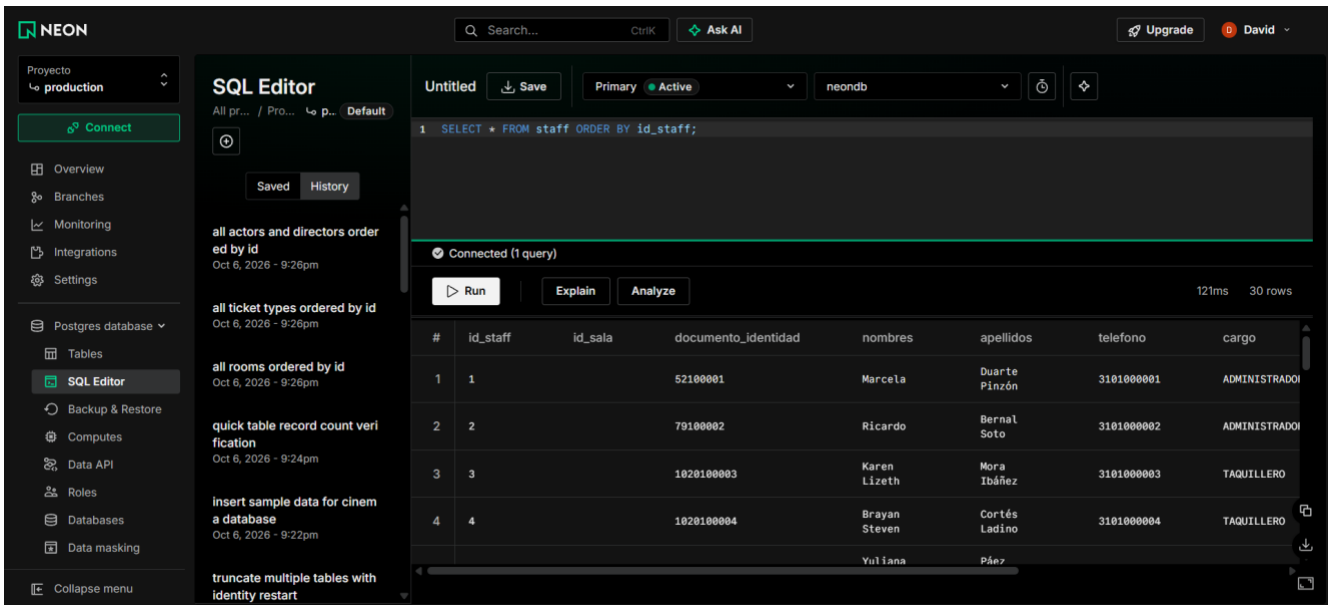


Figura 19. *Contenido de la tabla staff (captura 1 de 2).*

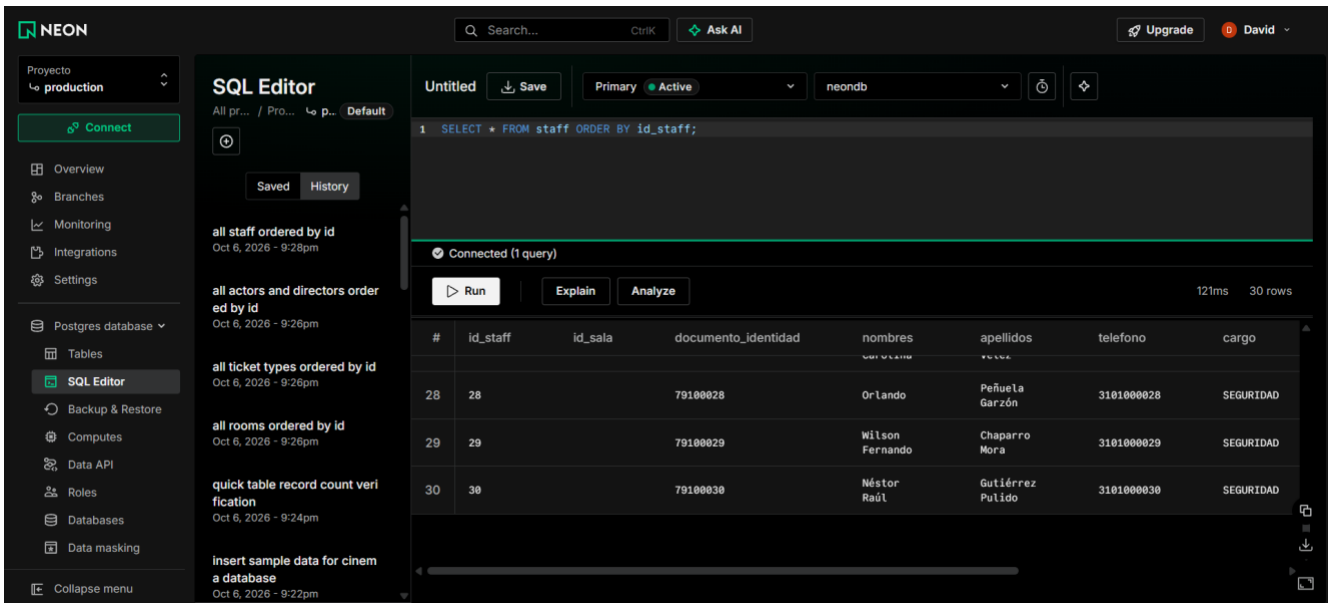


Figura 20. *Contenido de la tabla staff (captura 2 de 2).*

Contratos

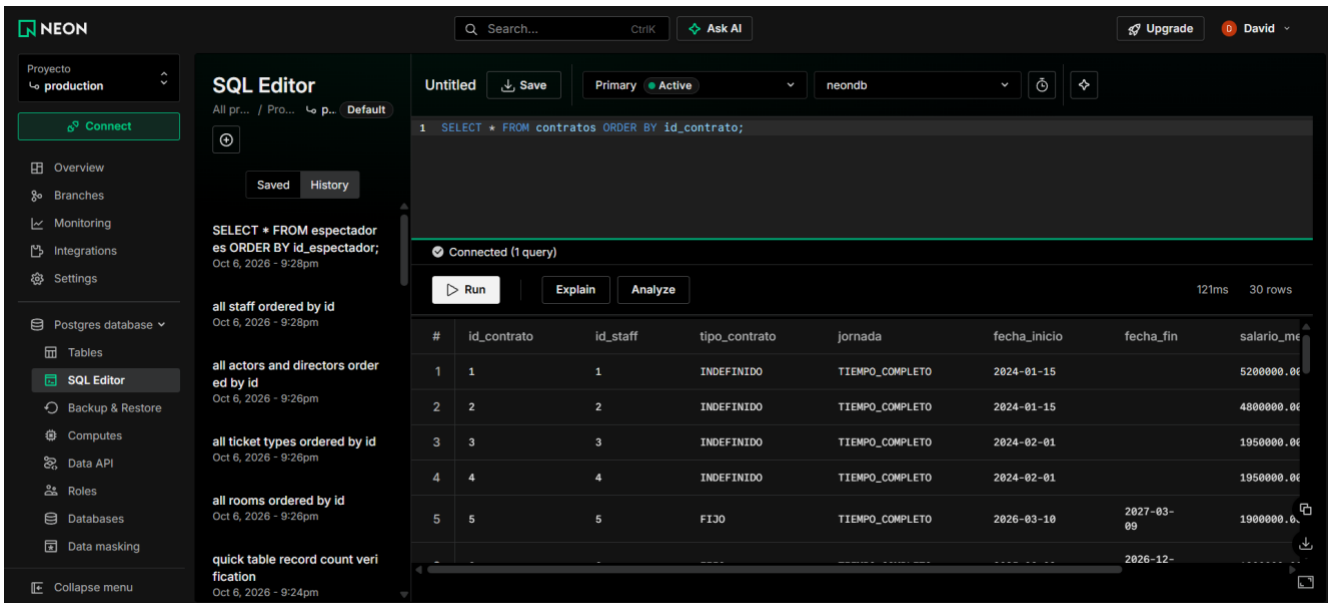


Figura 21. *Contenido de la tabla contratos (captura 1 de 2).*

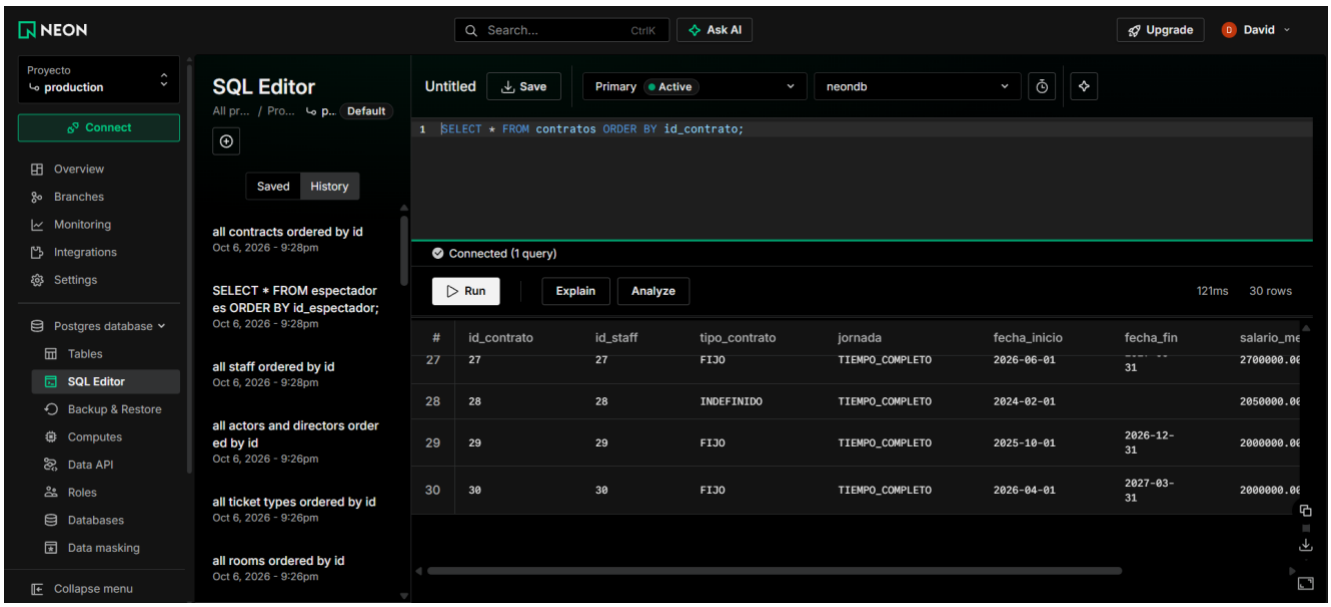


Figura 22. *Contenido de la tabla contratos (captura 2 de 2).*

Funciones

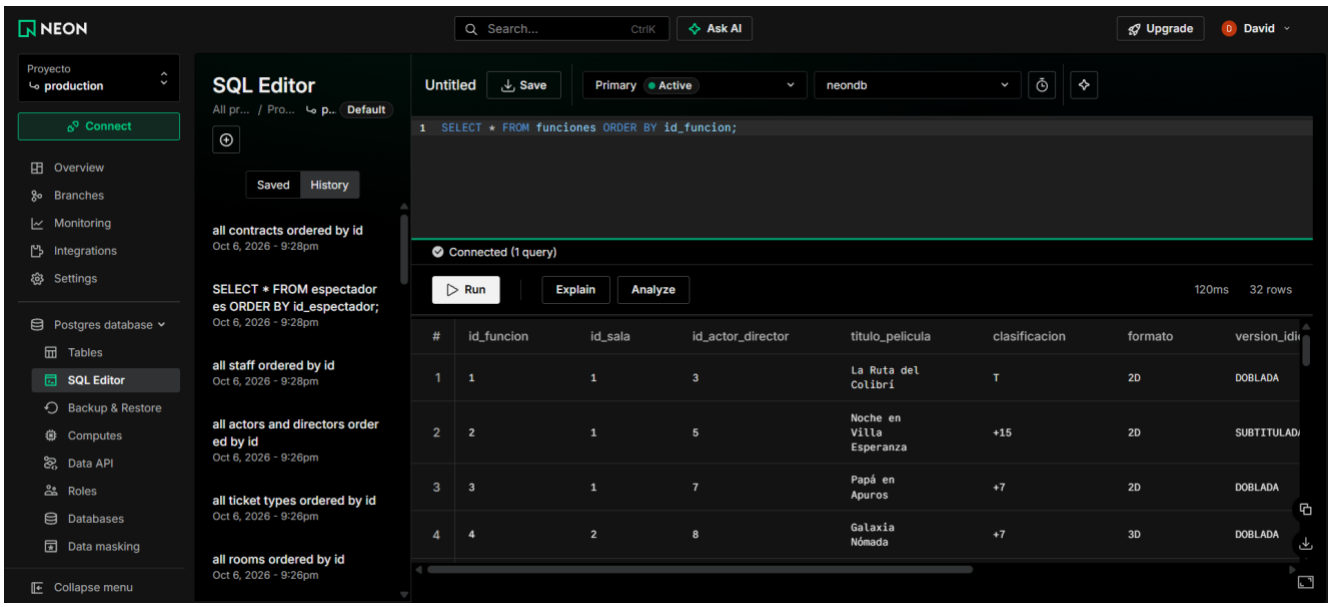


Figura 23. *Contenido de la tabla funciones (captura 1 de 2).*

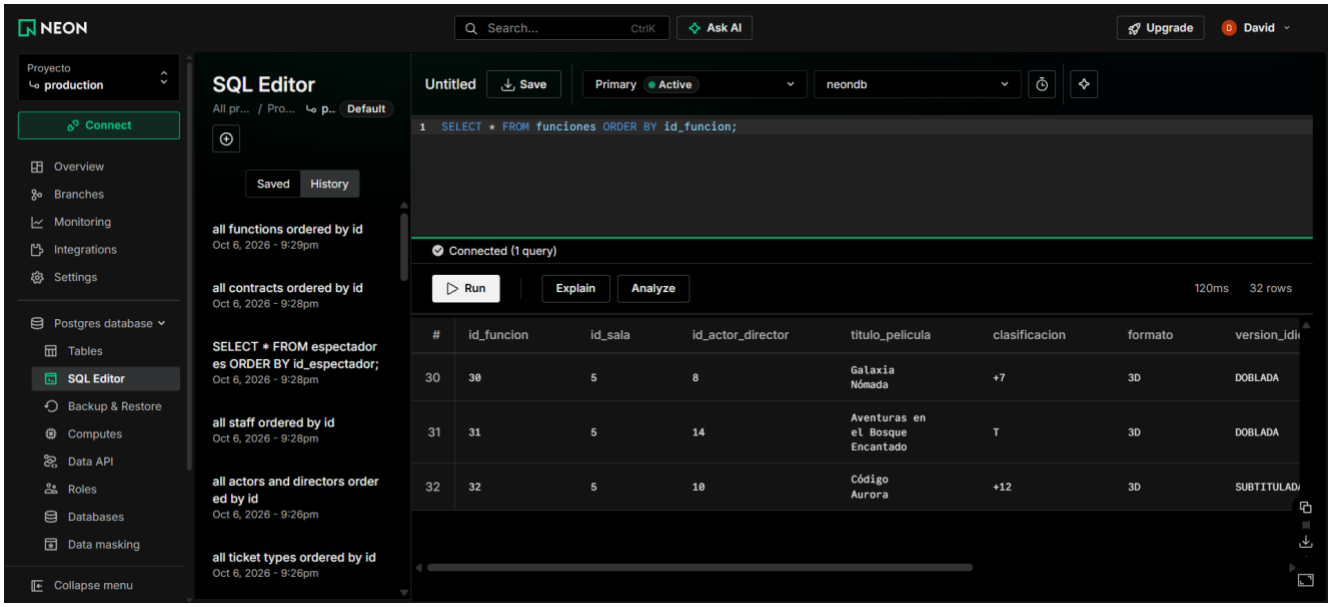


Figura 24. *Contenido de la tabla funciones (captura 2 de 2).*

Ventas

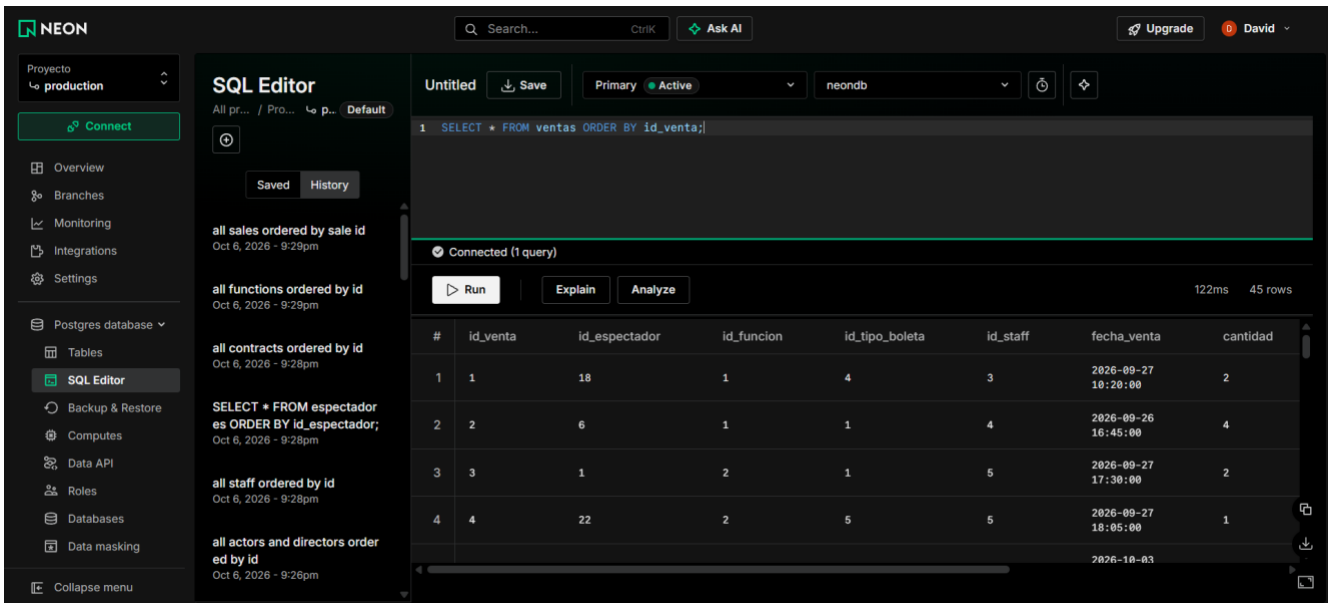


Figura 25. *Contenido de la tabla ventas (captura 1 de 2).*

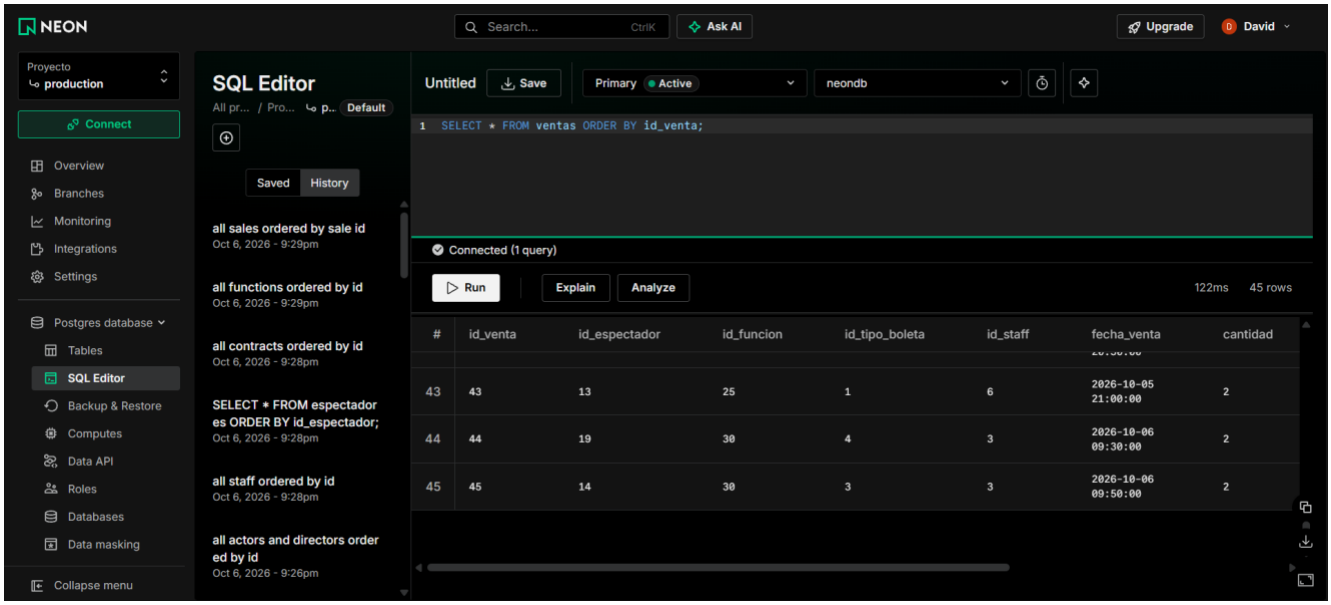


Figura 26. *Contenido de la tabla ventas (captura 2 de 2).*

3. Triggers

Se implementaron ocho triggers en PostgreSQL. Primero se describe cada trigger y después se muestran las pruebas que demuestran su funcionamiento.

3.1 Descripción de los triggers

Trigger 1: trg_validar_desactivar_sala

Este trigger se creó para evitar que se desactive una sala que aún tiene funciones programadas. La idea es que no se pueda dar de baja una sala si todavía se está utilizando para una función, ya que esto podría causar inconsistencias entre la información de las salas y las funciones programadas. Por lo tanto, el trigger, antes de hacer la actualización, mira si hay funciones asociadas a esa sala que todavía estén pendientes o en ejecución y si encuentra alguna cancela la operación y muestra un mensaje de error.

Trigger 2: trg_validar_desactivar_actor_director

Este trigger evita que se pueda desactivar un actor que aún tenga funciones programadas o en desarrollo. Esto es importante, ya que si una persona deja de estar activa en el sistema y aún aparece vinculada a funciones que se realizarán, la información se volvería inconsistente. Por eso, antes de cambiar el campo activo a false se miran las funciones asociadas y se bloquea la operación si todavía queda alguna pendiente.

Trigger 3: trg_validar_contrato

Este trigger evita que un mismo miembro del staff tenga dos contratos vigentes que coincidan en el mismo periodo de tiempo. Previo a la inserción o actualización de un contrato, se consultan los contratos existentes para ese trabajador y se comparan sus fechas de inicio y término. Si hay otro contrato vigente que se solape con el nuevo, la operación no está permitida. Con ello se pretende que la información laboral del personal quede organizada de forma coherente.

Trigger 4: trg_validar_desactivar_staff

Este trigger se encarga de que un miembro del staff no sea marcado como inactivo si aún tiene un contrato activo. El motivo de esta validación es que no tiene sentido que una persona aparezca como inactiva manteniendo dentro del sistema un contrato activo. Por lo tanto, el trigger, antes de cambiar el estado del empleado, consulta sus contratos y verifica que no exista uno vigente. Si lo encuentra, genera una excepción y no hace el cambio.

Trigger 5: trg_validar_fecha_espectador

Este trigger es usado para validar las fechas registradas de los espectadores. Su función es evitar que la fecha de nacimiento sea posterior a la fecha en que la persona fue

registrada en el sistema, porque sería un estado incorrecto. La validación se hace antes de insertar o modificar un espectador y si las fechas no cumplen con esta condición, el trigger detiene la operación y muestra un mensaje indicando cuál es el problema.

Trigger 6: trg_validar_precio_boleta

Este trigger regula los cambios de precio de los tipos de boleta que ya están inactivos. Si una tipología de entrada está marcada como inactiva no se puede modificar su precio, porque esa tipología de entrada no debería estar disponible para nuevas operaciones. De esta manera se evita alterar información de un tipo de boleta ya retirada del sistema y se mantiene una mayor consistencia en los datos históricos.

Trigger 7: trg_validar_horario_funcion

Este trigger se creó para evitar que en una misma sala se programen dos funciones en el mismo horario. Antes de registrar o modificar una función, se comparan sus horarios con los de las demás funciones que utilizan la misma sala. Si se produce un solapamiento entre la hora de inicio y la hora de fin, el sistema rechaza la operación. Esto evita que una misma sala aparezca programada para dos películas distintas al mismo tiempo.

Trigger 8: trg_validar_capacidad_venta

Este trigger controla que la cantidad de boletas vendidas para una función no sea mayor a la capacidad de la sala en la que se realiza. Antes de registrar una venta nueva, se consulta la capacidad de la sala y se suman las boletas ya vendidas para esa función. El trigger impide que se realice la nueva venta si esta supera el límite permitido. Por otra parte, no se consideran las ventas que se encuentran anuladas, ya que dichas entradas ya no representan ocupación real de sala.

3.2 Pruebas

Cada prueba intenta realizar una operación que el trigger debe rechazar. El bloque anónimo captura la excepción y muestra el mensaje «OK: Trigger N funcionando» junto con el motivo del bloqueo. Las capturas muestran ese aviso en pantalla.

Prueba 1: Intentar desactivar una sala que tiene funciones

La sala 1 tiene funciones programadas.

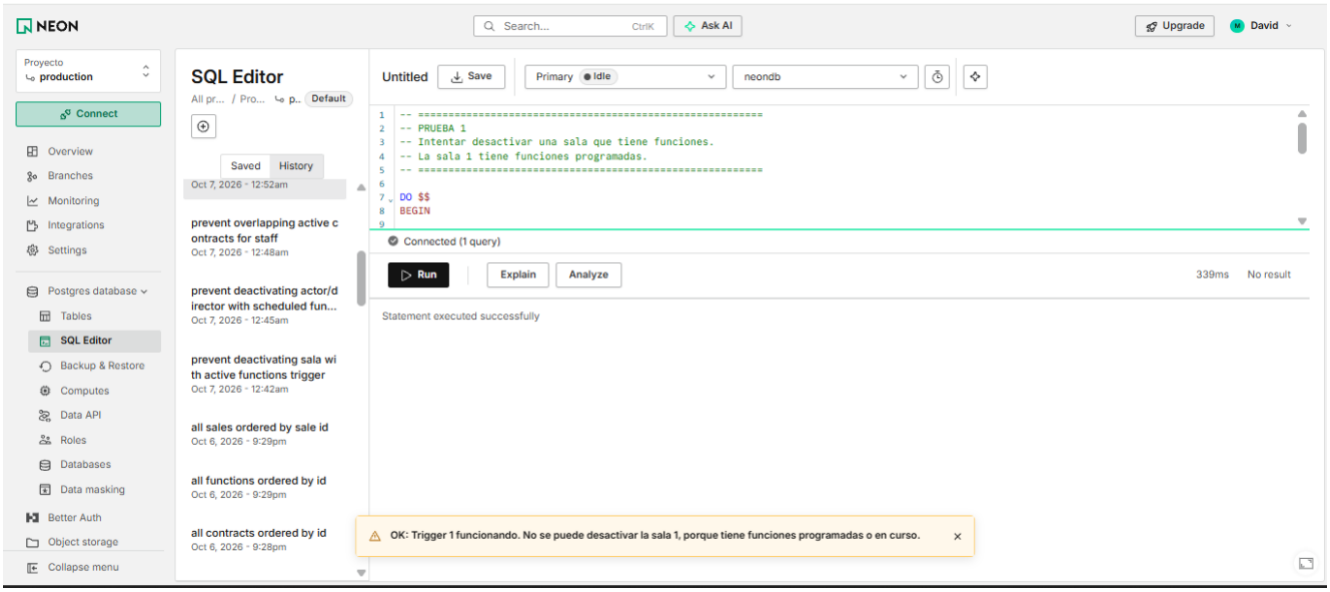


Figura 27. *Ejecución de la prueba 1 del trigger trg_validar_desactivar_sala.*

Resultado: el trigger bloqueó la operación y mostró el aviso No se puede desactivar la sala 1, porque tiene funciones programadas o en curso.

Prueba 2: Intentar desactivar un actor/director que tiene una función programada

El actor/director 16 participa en la función 17.

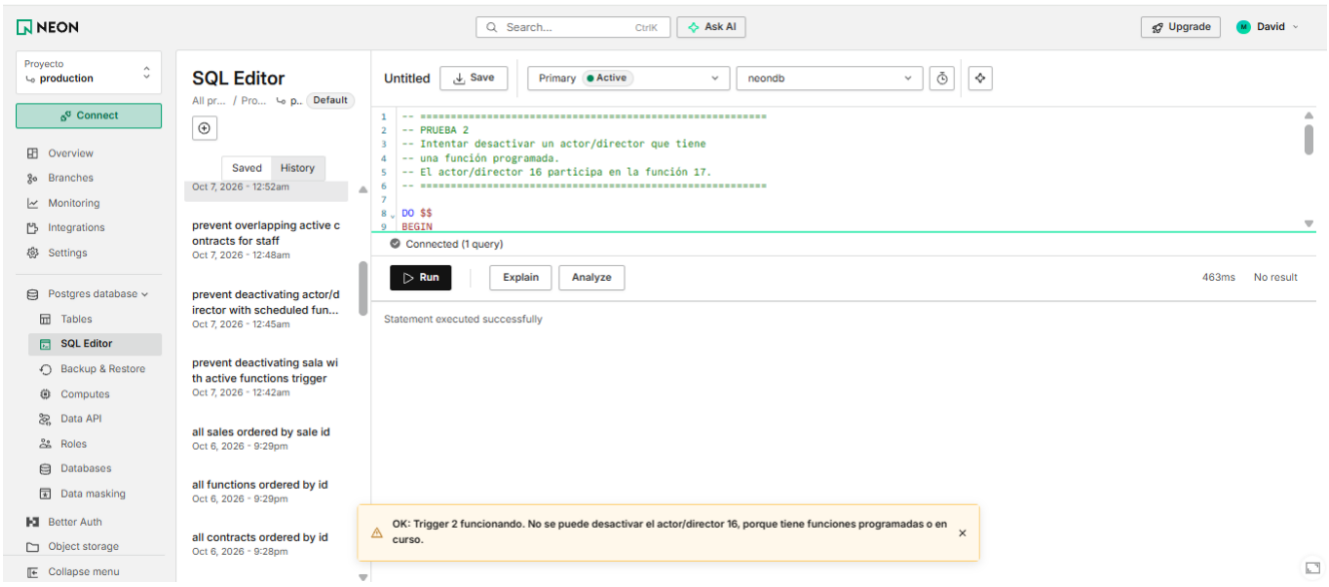


Figura 28. *Ejecución de la prueba 2 del trigger trg_validar_desactivar_actor_director.*

Resultado: el trigger bloqueó la operación y mostró el aviso No se puede desactivar el actor/director 16, porque tiene funciones programadas o en curso.

Prueba 3: Intentar crear un contrato vigente que se cruza con el contrato vigente del staff 1

Se inserta el contrato 31 para el staff 1, del 2026-01-01 al 2026-12-31.

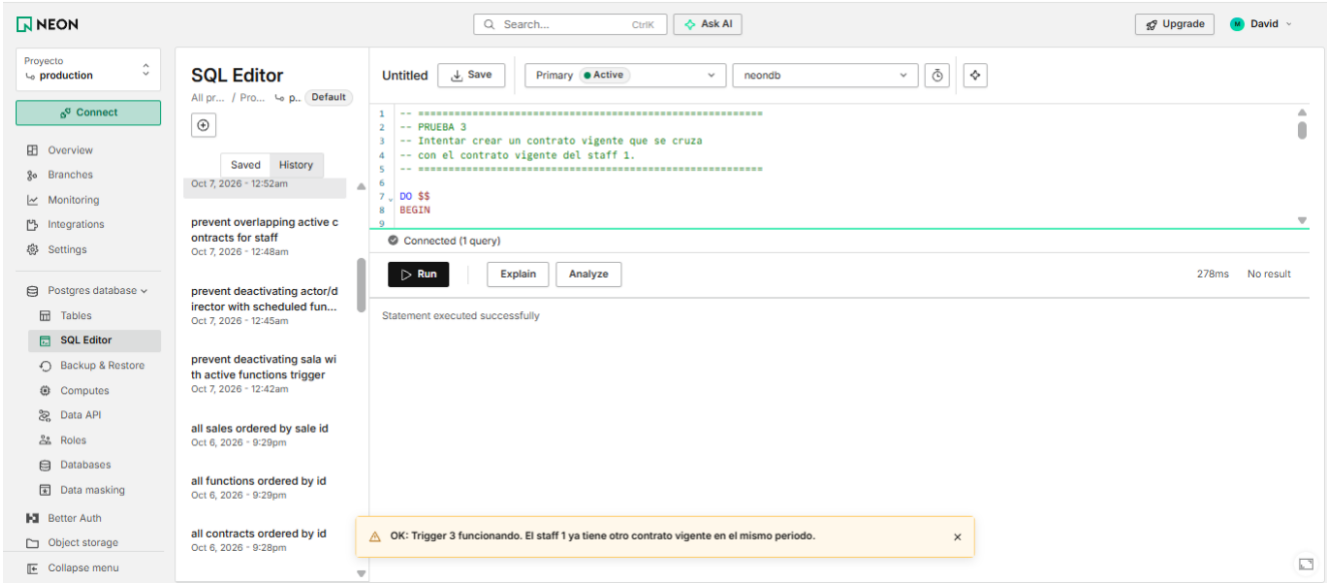


Figura 29. *Ejecución de la prueba 3 del trigger trg_validar_contrato.*

Resultado: el trigger bloqueó la operación y mostró el aviso El staff 1 ya tiene otro contrato vigente en el mismo periodo.

Prueba 4: Intentar desactivar un miembro del staff que tiene un contrato vigente

Se intenta poner activo = FALSE al staff 1.

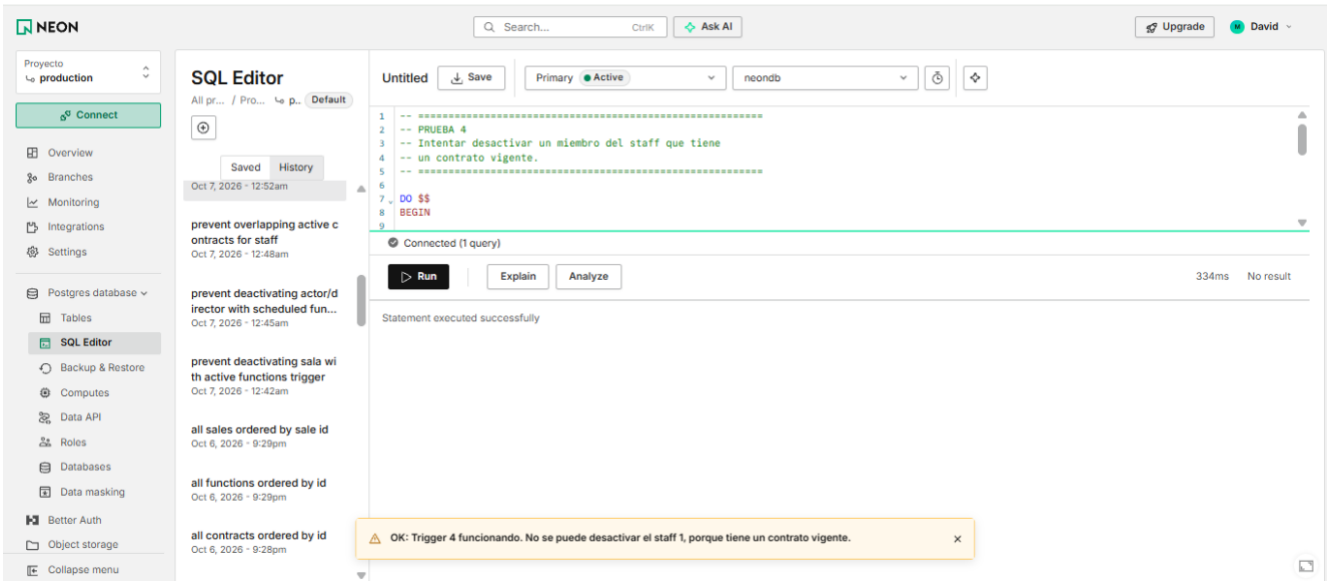


Figura 30. *Ejecución de la prueba 4 del trigger trg_validar_desactivar_staff.*

Resultado: el trigger bloqueó la operación y mostró el aviso No se puede desactivar el staff 1, porque tiene un contrato vigente.

Prueba 5: Intentar registrar un espectador cuya fecha de nacimiento es posterior a la fecha de registro

Fecha de nacimiento 2027-01-01 y fecha de registro 2026-10-07.

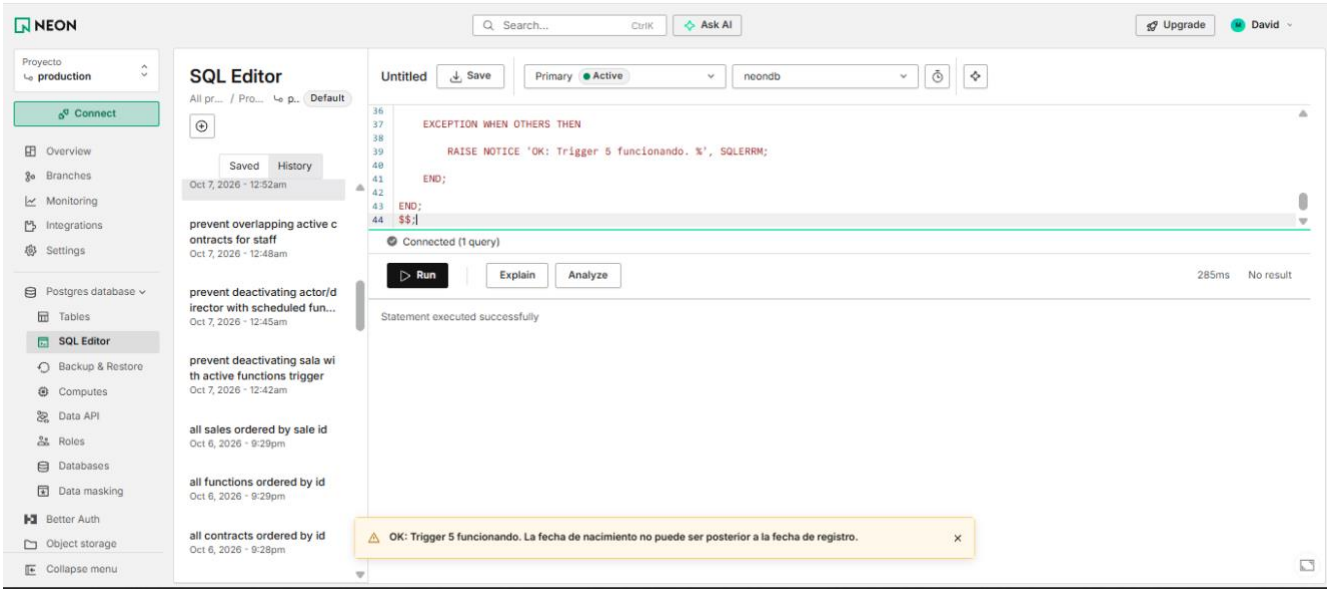


Figura 31. *Ejecución de la prueba 5 del trigger trg_validar_fecha_espectador.*

Resultado: el trigger bloqueó la operación y mostró el aviso La fecha de nacimiento no puede ser posterior a la fecha de registro.

Prueba 6: Intentar modificar el precio de una boleta inactiva

Se desactiva el tipo de boleta 1 y luego se intenta cambiar su precio a 19000.

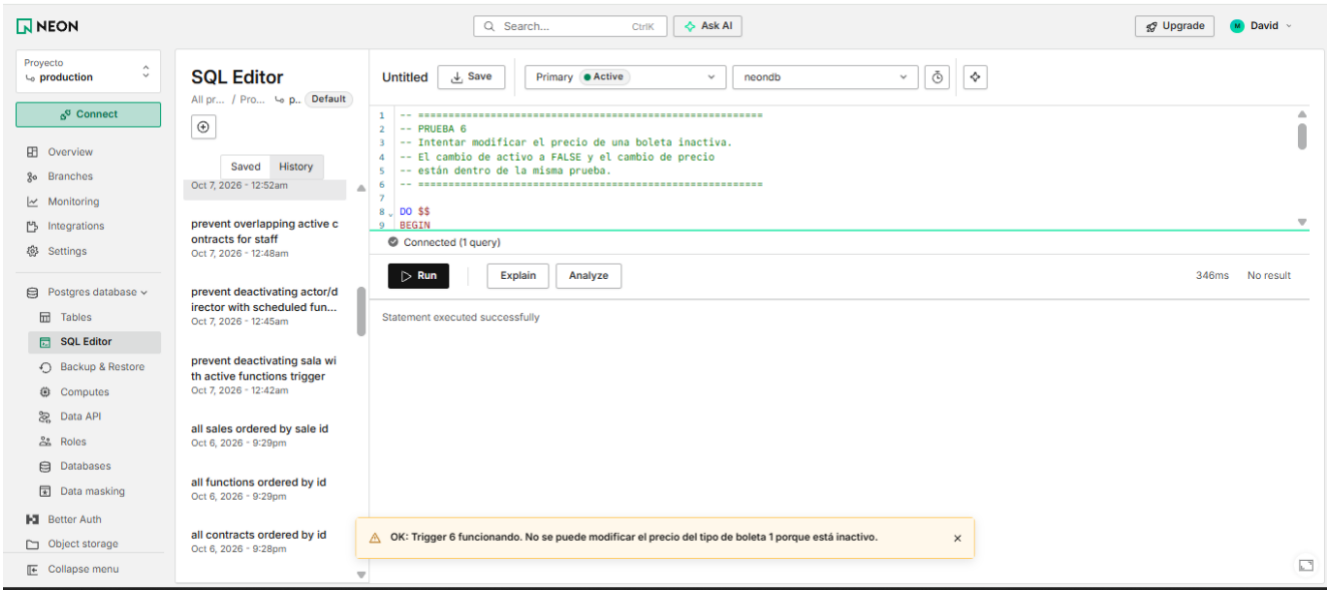


Figura 32. *Ejecución de la prueba 6 del trigger trg_validar_precio_boleta.*

Resultado: el trigger bloqueó la operación y mostró el aviso No se puede modificar el precio del tipo de boleta 1 porque está inactivo.

Prueba 7: Intentar crear una función cuyo horario se cruza

La función 33 se programa en la sala 1 de 17:30 a 19:00, cruzada con la función 17.

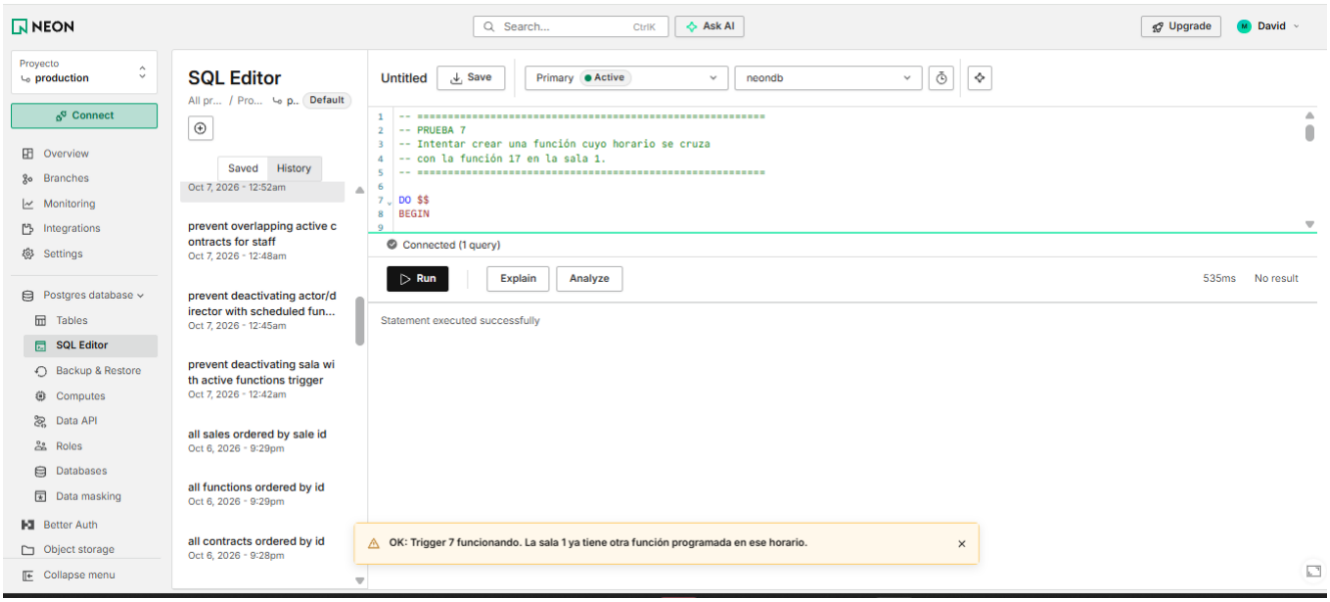


Figura 33. *Ejecución de la prueba 7 del trigger trg_validar_horario_funcion.*

Resultado: el trigger bloqueó la operación y mostró el aviso La sala 1 ya tiene otra función programada en ese horario.

Prueba 8: Intentar superar la capacidad de la sala

Se registran cuatro ventas de 10 boletas para la función 24, cuya sala tiene capacidad para 40.

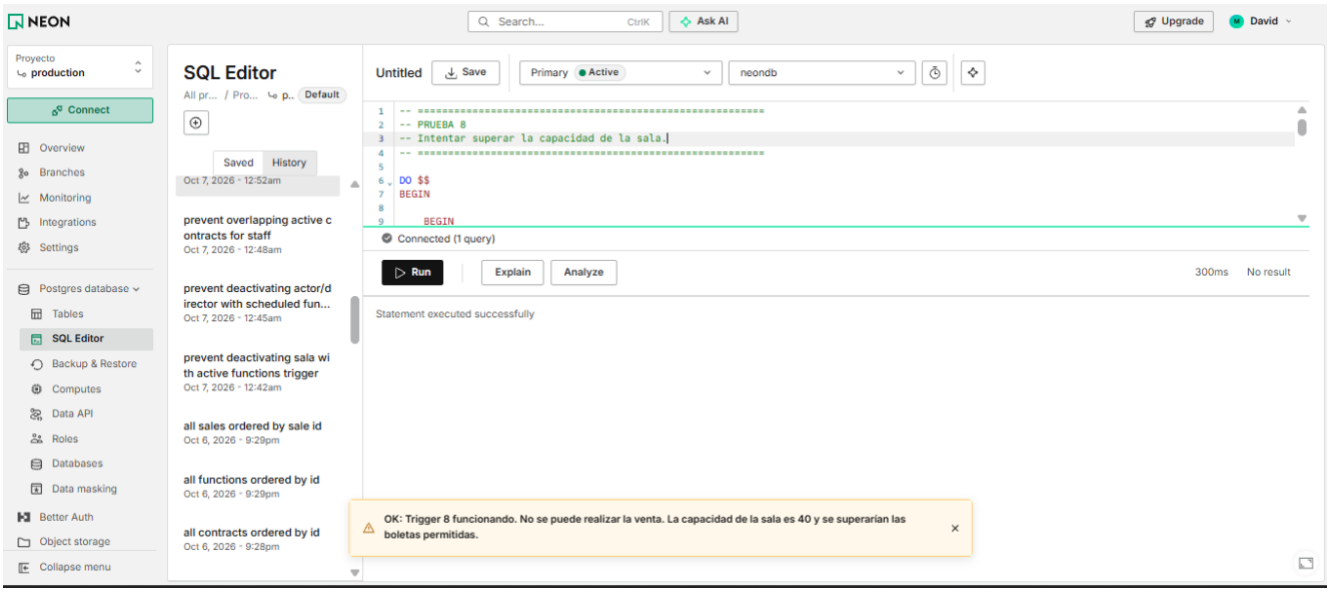


Figura 34. *Ejecución de la prueba 8 del trigger trg_validar_capacidad_venta.*

Resultado: el trigger bloqueó la operación y mostró el aviso No se puede realizar la venta. La capacidad de la sala es 40 y se superarían las boletas permitidas.

4. Procedimientos

Se crearon cuatro procedimientos almacenados: uno de cálculo, uno de reporte con agregación, uno de operación y uno adicional por ser grupo de cinco. Primero se muestra la creación de cada uno y luego las pruebas de ejecución.

4.1 Creación de los procedimientos

Captura 1: procedimiento 1 · sp_calcular_ocupacion (cálculo)

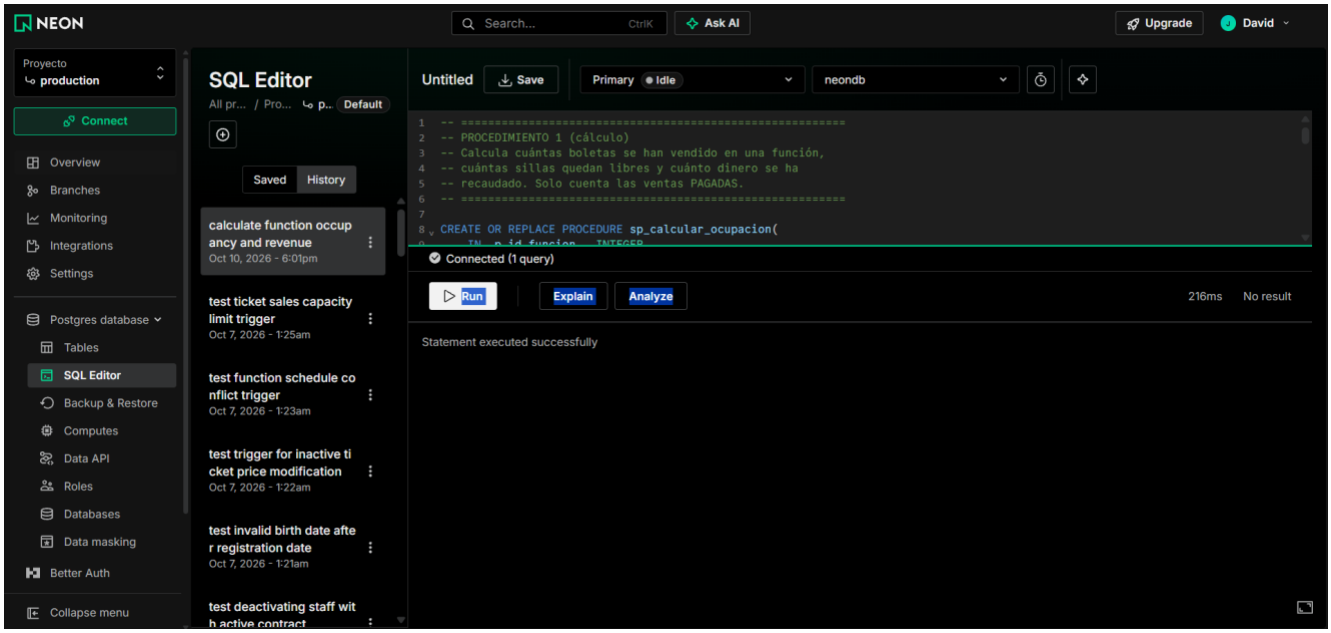


Figura 35. *Creación del procedimiento sp_calcular_ocupacion.*

En esta captura se está ejecutando el script que crea el procedimiento sp_calcular_ocupacion, que es el de cálculo. Toma como parámetro el id de una función y devuelve la cantidad de boletas vendidas, la cantidad de sillas libres en la sala y el dinero recaudado, contando sólo las ventas pagadas. Si la función no se encuentra, se producirá un error con RAISE EXCEPTION, y el bloque EXCEPTION recogerá cualquier otro error que pueda ocurrir. Neon muestra “Statement executed successfully”, lo que significa que el procedimiento se creó correctamente, sin errores. Para el cine sirve para saber qué tan llena va a estar una función.

Captura 2: procedimiento 2 · sp_reporte_ventas_pelicula (reporte con agregación)

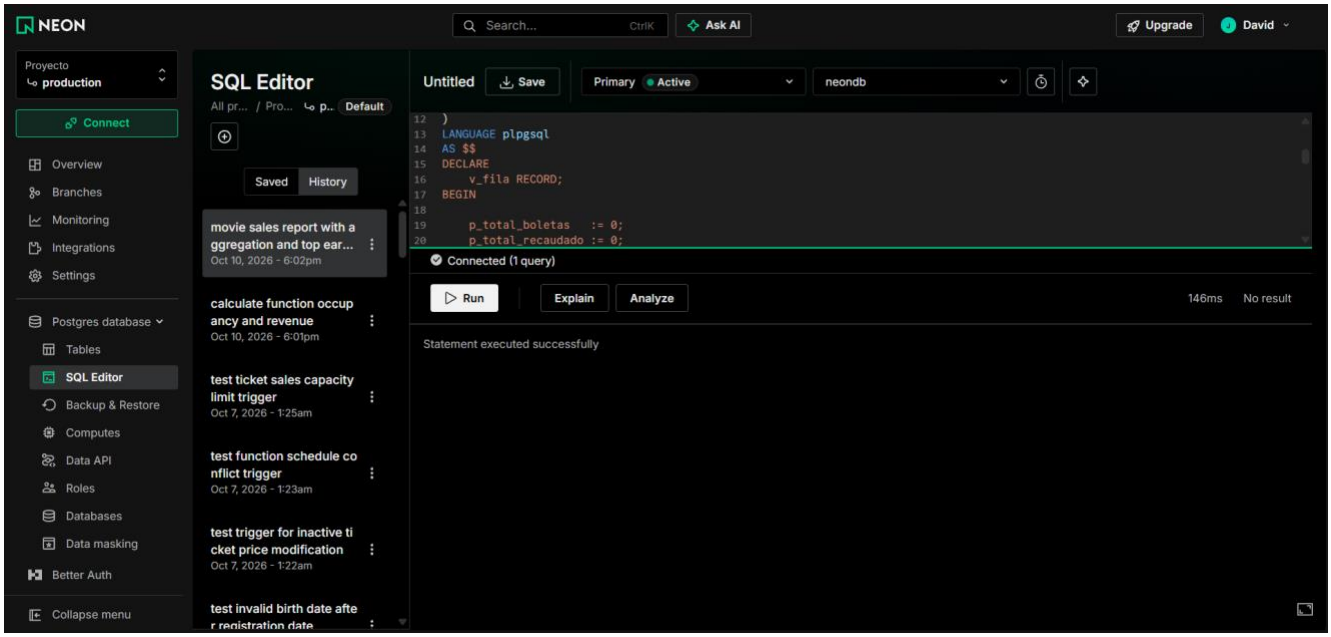


Figura 36. *Creación del procedimiento sp_reporte_ventas_pelicula.*

Aquí se crea el procedimiento sp_reporte_ventas_pelicula, el de reporte con agregación. Utiliza SUM y GROUP BY para sumar las boletas vendidas y el dinero recaudado de cada película. Al final imprime el total de boletas vendidas, el total recaudado y la película que más recaudó. If there are no paid sales, throw an error with RAISE EXCEPTION. La ejecución terminó con “Statement executed successfully”, por lo que se creó correctamente. El cine utiliza este reporte para saber cuáles son las películas más vendidas.

Captura 3: procedimiento 3 · sp_anular_venta (operación)

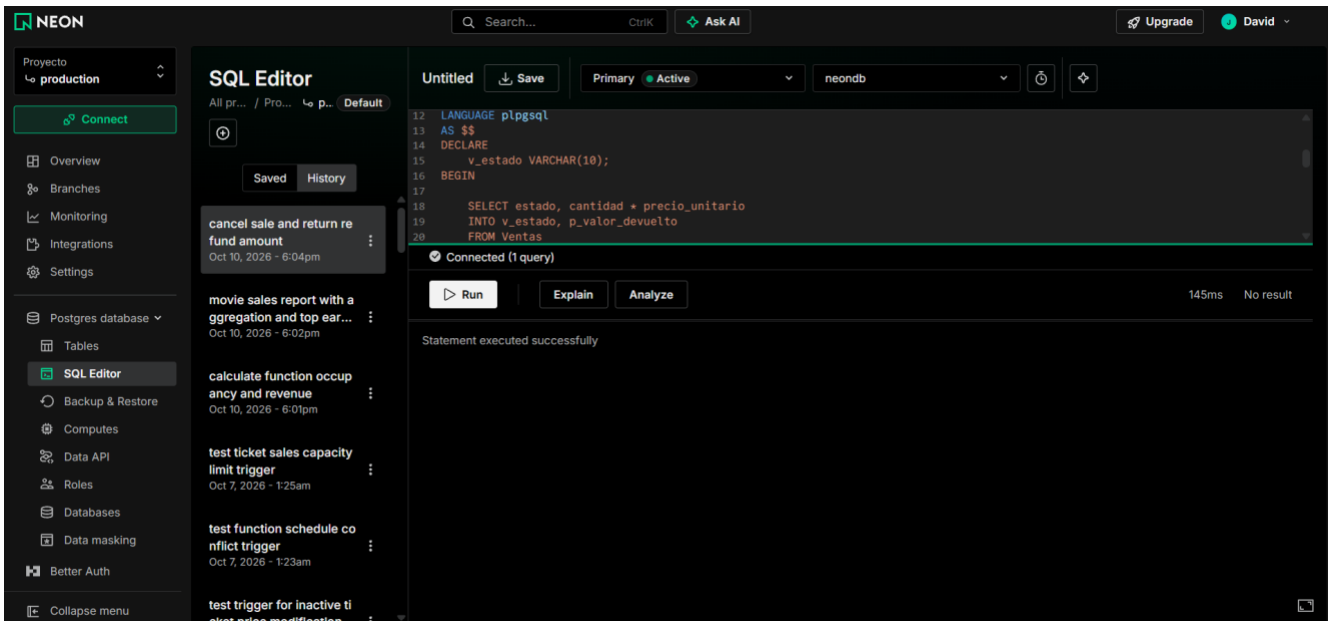


Figura 37. *Creación del procedimiento sp_anular_venta.*

Se crea sp_anular_venta, el procedimiento de operacion. Recibe el id de una venta y comprueba que exista y que no esté anulada. Después la pasa a estado ANULADA y devuelve el valor que hay que devolverle al espectador. Si no existe la venta o ya estaba anulada se produce un error con RAISE EXCEPTION. Neon nos confirma que el procedimiento se ha creado sin problemas. Sirve para los casos donde un cliente solicita devolver su compra.

Captura 4: procedimiento 4 · sp_registrar_venta (adicional)

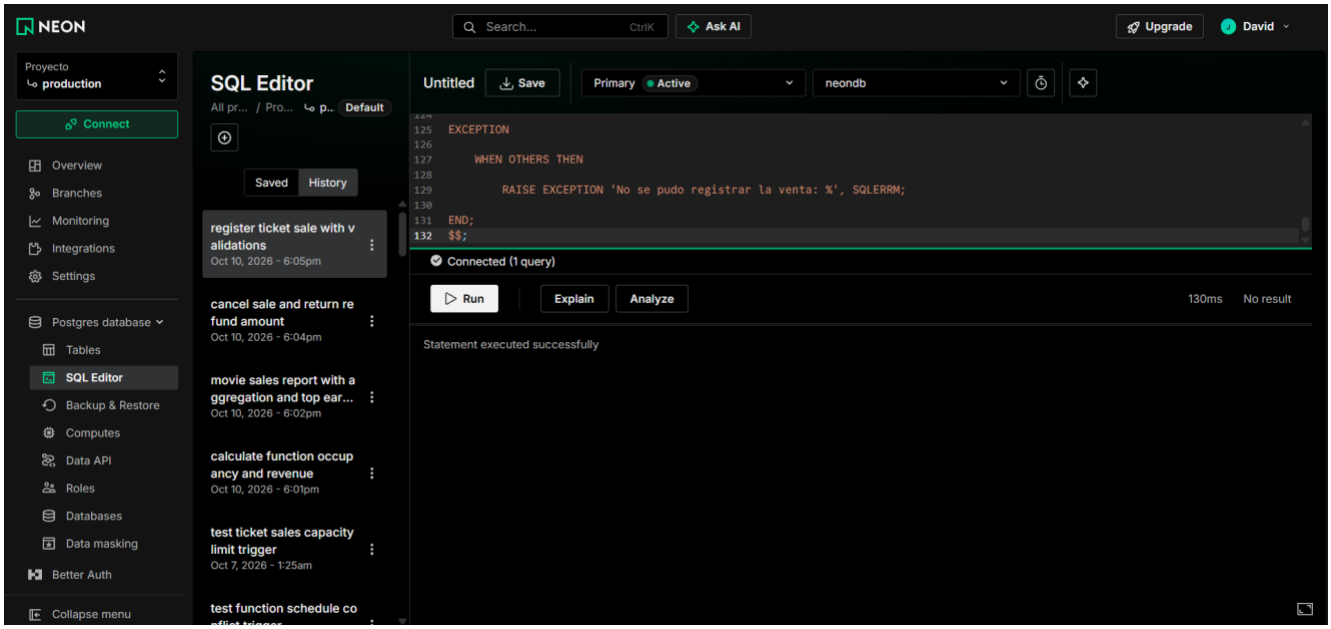


Figura 38. *Creación del procedimiento sp_registrar_venta.*

Se crea sp_registrar_venta, procedimiento adicional por ser grupo de 5. Este proceso hace la venta entera y antes de guardar, controla que exista el espectador, que exista la función y que esté programada, que el espectador tenga la edad que pide la clasificacion de la película, que exista y esté activo el tipo de boleta y el staff, y que haya sillas suficientes. Si alguna validación falla se produce un RAISE EXCEPTION y no se guarda nada. El precio se toma de la tabla Tipos_Boleta, para que no se pueda escribir uno diferente. Se ve en la captura que se ha creado correctamente.

4.2 Pruebas de los procedimientos

Captura 5: prueba del procedimiento 1

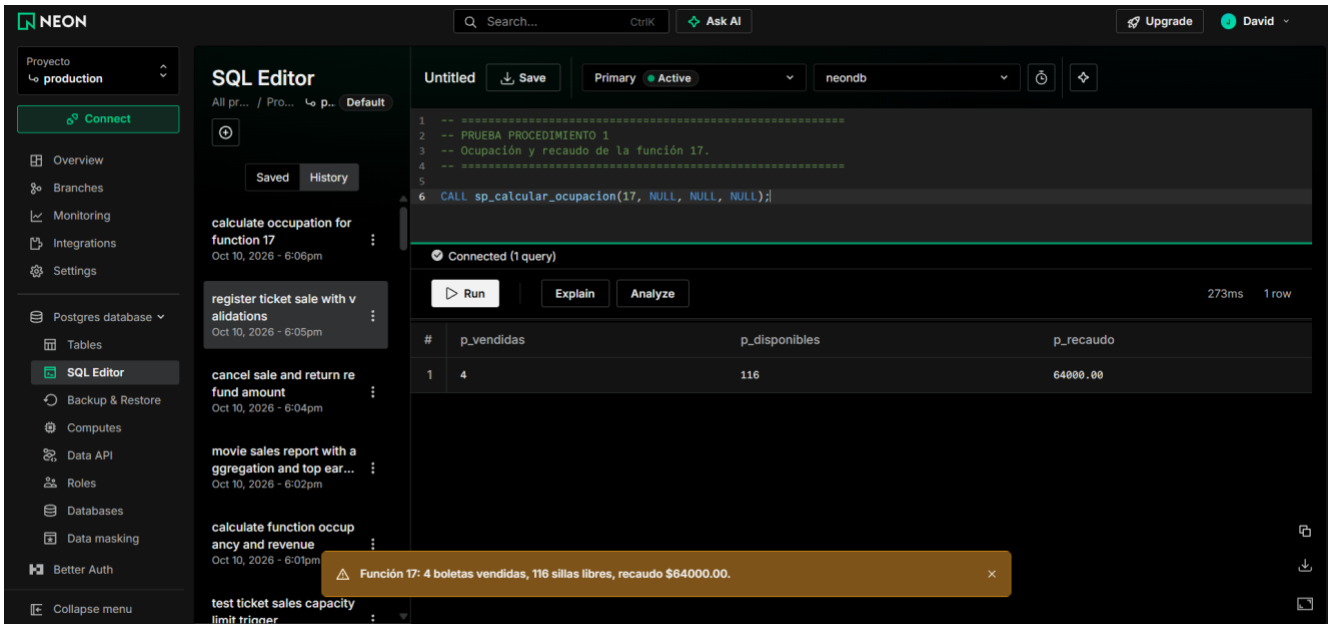


Figura 39. *Prueba de sp_calcular_ocupacion para la función 17.*

Para calcular la ocupación de la función 17 se ejecuta CALL sp_calcular_ocupacion(17, NULL, NULL, NULL). El resultado indica que se vendieron 4 boletas, quedaron 116 sillas libres y se recaudó $64.000. A continuación también aparece el mensaje del RAISE NOTICE con el mismo resumen. Con esto se comprueba que el procedimiento suma bien las ventas pagadas del función y que resta las vendidas de la capacidad del local que son 120 sillas.

Captura 6: prueba del procedimiento 2

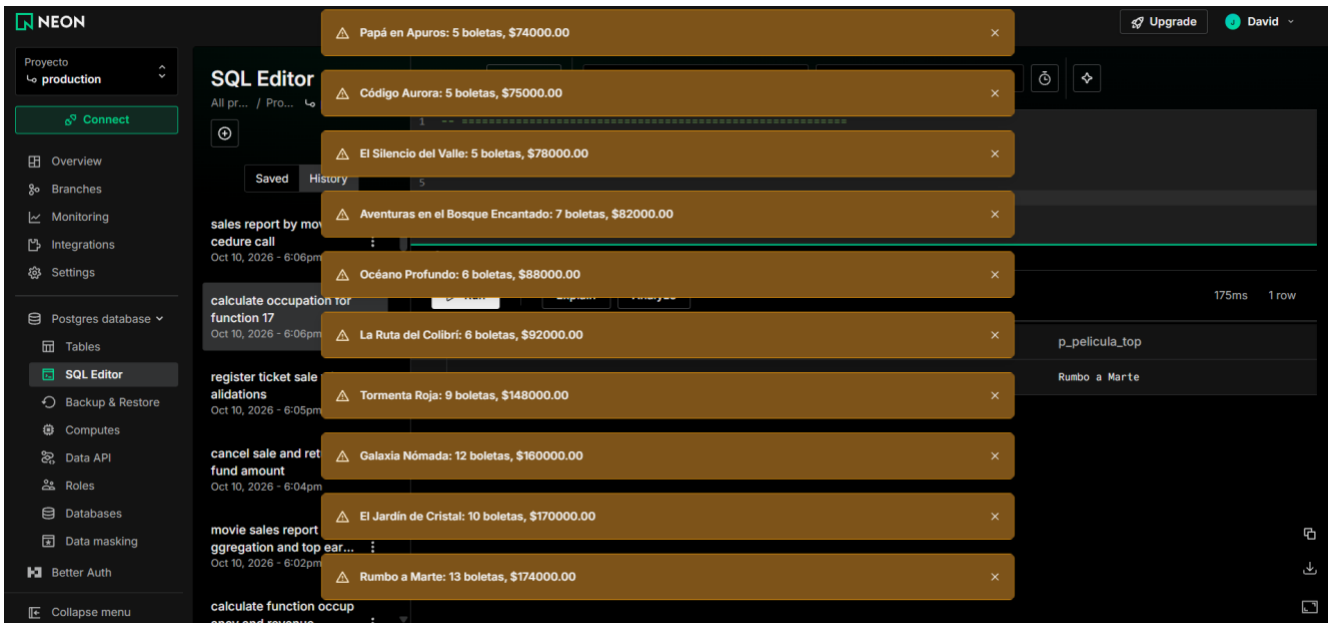


Figura 40. *Prueba de sp_reporte_ventas_pelicula.*

Se ejecuta CALL sp_reporte_ventas_pelicula (NULL, NULL, NULL) Los mensajes de RAISE NOTICE, muestran las taquillas vendidas y el dinero recaudado de cada película. Por ejemplo, Galaxia Nómada tiene 12 boletas y $160.000, El Jardín de Cristal tiene 10 boletas y $170.000. El resultado muestra que la película más taquillera es Rumbo a Marte, con 13 boletas y $174.000. Con esto comprobamos que la agregación por película funciona.

Captura 7: prueba del procedimiento 4

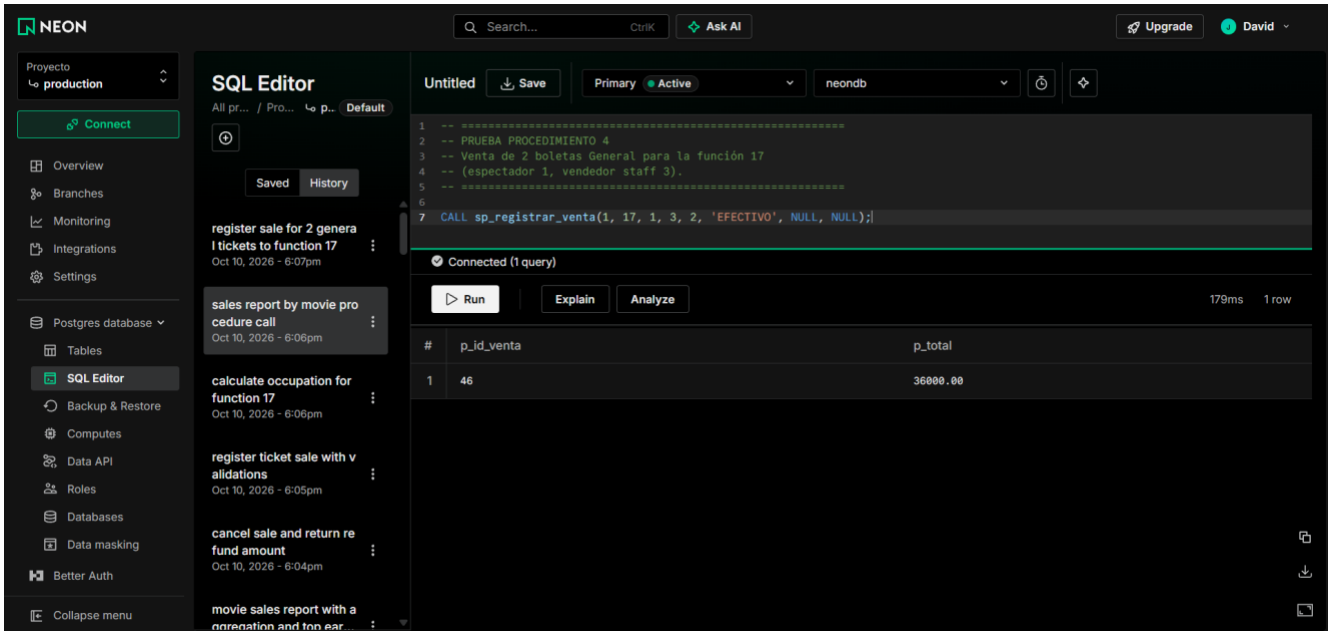


Figura 41. *Prueba de sp_registrar_venta: venta de 2 boletas General.*

Se ejecuta CALL sp_registrar_venta (1,17,1,3,2,'EFECTIVO',NULL,NULL). Esa llamada vende al espectador 1 2 boletas de tipo General para la función 17, con pago en efectivo, y fue atendida por el staff 3. Con las validaciones realizadas se procedió a registrar la venta id 46 por un total de $36.000, correspondiente a 2 boletas de $18.000. Esto demuestra que el proceso hace todo el flujo de la venta y coge el precio de la tabla Tipos_Boleta.

Captura 8: prueba del procedimiento 3

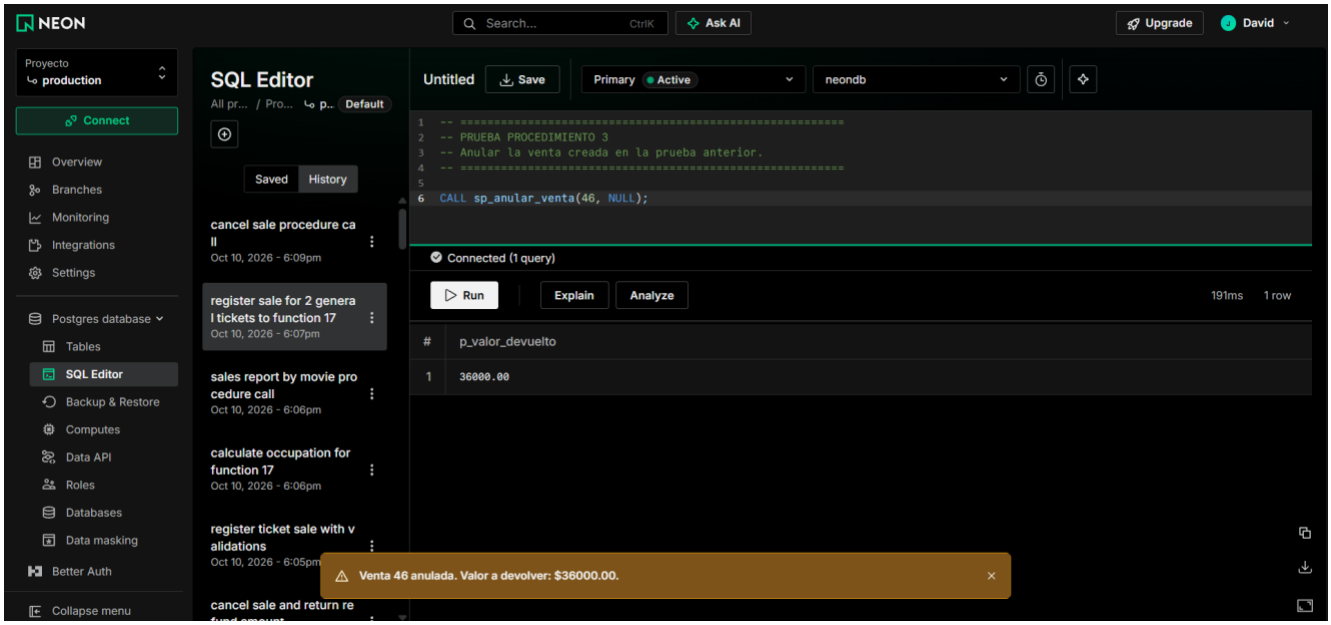


Figura 42. *Prueba de sp_anular_venta sobre la venta creada en la prueba anterior.*

Para anular la venta creada en la prueba anterior se ejecuta CALL sp_anular_venta(46, NULL). El resultado devuelve un valor de $36.000, y el mensaje de RAISE NOTICE confirma que la venta 46 fue anulada. De este modo comprobamos que el procedimiento cambia el estado de la venta a ANULADA y calcula bien el dinero que se le devuelve al espectador.

5. Consultas SQL

Se ejecutaron cuatro consultas sobre los datos cargados. Cada una se presenta con la captura de su resultado, las tablas implicadas y su propósito de negocio.

Consulta 1

#	metodo_pago	total_transacciones	total_boletas_vendidas	recaudo_total
1	TARJETA	20	51	847000.00
2	EFECTIVO	17	34	400000.00
3	TRANSFERENCIA	7	16	227000.00

Figura 43. *Ejecución de la consulta 1.*

Tablas implicadas: ventas
Propósito de negocio: Esta consulta agrupa las ventas efectivas (PAGADA) según el método de pago utilizado (ej. efectivo, tarjeta débito, crédito). Permite a la administración del cine identificar cuáles son las pasarelas o medios de pago más utilizados por los espectadores y qué volumen de dinero ingresa a través de cada canal, facilitando la toma de decisiones financieras.

Consulta 2

#	id_funcion	titulo_pelicula	clasificacion	formato	nombre	capacidad_sala	fecha_hora_inicio	estado_funcion
1	17	Latidos	+12	2D	Roble	120	2026-10-07 17:00:00	PROGRAMADA
2	18	Duelo en la Frontera	+15	2D	Roble	120	2026-10-07 20:00:00	PROGRAMADA
3	21	Rumbo a Marte	+7	3D	Cedro	100	2026-10-08 17:30:00	PROGRAMADA
4	22	Océano Profundo	+12	3D	Cedro	100	2026-10-08 20:00:00	PROGRAMADA
5	24	Tormenta Roja	+18	2D	Guayacán	40	2026-10-09 20:30:00	PROGRAMADA
6	27	El Silencio del Valle	+18	2D	Ceiba	150	2026-10-09 21:00:00	PROGRAMADA
7	19	La Ruta del Colibrí	T	2D	Roble	120	2026-10-10 14:00:00	PROGRAMADA
8	30	Galaxia Nómada	+7	3D	Sauce	90	2026-10-10 15:00:00	PROGRAMADA
9	20	Papá en Apuros	+7	2D	Roble	120	2026-10-10 16:30:00	PROGRAMADA
10	25	El Jardín de Cristal	+12	2D	Guayacán	40	2026-10-10 19:00:00	PROGRAMADA
11	28	Mi Abuela la Hechicera	T	2D	Ceiba	150	2026-10-11 11:00:00	PROGRAMADA
12	31	Aventuras en el Bosque Encantado	T	3D	Sauce	90	2026-10-11 11:30:00	PROGRAMADA
13	23	Dragones del Alba	+7	3D	Cedro	100	2026-10-11 15:00:00	PROGRAMADA
14	29	Cenizas de Julio	+15	2D	Ceiba	150	2026-10-11 18:00:00	PROGRAMADA
15	32	Código Aurora	+12	3D	Sauce	90	2026-10-12 19:00:00	PROGRAMADA
16	26	Código Aurora	+12	3D	Guayacán	40	2026-10-12 19:30:00	PROGRAMADA

Figura 44. *Ejecución de la consulta 2.*

Tablas implicadas: funciones, salas
Propósito de negocio: Mediante un INNER JOIN entre las funciones programadas y la infraestructura de las salas, esta consulta genera una vista detallada de la cartelera activa. Ayuda al equipo de operaciones y logística a supervisar qué películas están proyectándose en qué salas y cuál es la capacidad máxima disponible para la venta.

Consulta 3

#	id_espectador	nombre_completo	correo	id_venta	cantidad	precio_unitario
1	6	Sebastián Ramírez Cruz	sebastian.ramirez@correo.example	2	4	18000.00
2	8	Andrés Castro Villamil	andres.castro@correo.example	8	4	18000.00
3	28	Esteban Murillo Gaitán	esteban.murillo@correo.example	14	4	13000.00
4	2	Santiago Rojas Mejía	santiago.rojas@correo.example	17	4	18000.00
5	5	Isabella Torres Quintero	isabella.torres@correo.example	30	4	14000.00
6	8	Andrés Castro Villamil	andres.castro@correo.example	40	3	18000.00
7	32	Julián Barrera Nieto	julian.barrera@correo.example	34	3	14000.00
8	30	Miguel Ángel Rey Salamanca	miguelangel.rey@correo.example	22	3	13000.00
9	7	Laura Díaz Martínez	laura.diaz@correo.example	5	3	18000.00
10	11	Mariana Silva Arango	mariana.silva@correo.example	26	3	18000.00
11	10	Juan Pablo Moreno Gil	juanpablo.moreno@correo.example	11	3	14000.00
12	21	Salomé Rincón Escobar	salome.rincon@correo.example	13	3	10000.00

Figura 45. *Ejecución de la consulta 3.*

Tablas implicadas: espectadores, ventas
Propósito de negocio: Identifica a los clientes que realizan compras de volumen superior al promedio general de boletas por transacción registradas en el sistema. Es fundamental para estrategias de fidelización de clientes y campañas de marketing dirigidas a espectadores frecuentes o corporativos.

Consulta 4 (adicional)

#	id_funcion	titulo_pelicula	formato	nombre	recaudo_total	ranking_formato	ranking_general
1	7	El Jardín de Cristal	2D	Guayacán	134000.00	7	13
2	4	Galaxia Nómada	3D	Cedro	116000.00	7	14
3	13	Rumbo a Marte	3D	Sauce	100000.00	8	15
4	1	La Ruta del Colibrí	2D	Roble	92000.00	8	16
5	5	Océano Profundo	3D	Cedro	88000.00	9	17
6	8	Tormenta Roja	2D	Guayacán	86000.00	9	18
7	6	Aventuras en el Bosque Encantado	3D	Cedro	82000.00	10	19
8	11	El Silencio del Valle	2D	Ceiba	78000.00	10	20
9	9	Código Aurora	3D	Guayacán	75000.00	11	21
10	21	Rumbo a Marte	3D	Cedro	74000.00	12	23
11	3	Papá en Apuros	2D	Roble	74000.00	11	22
12	10	Cenizas de Julio	2D	Ceiba	70000.00	12	24
13	14	Dragones del Alba	3D	Sauce	66000.00	13	25
14	17	Latidos	2D	Roble	64000.00	13	26
15	24	Tormenta Roja	2D	Guayacán	62000.00	14	27
16	2	Noche en Villa Esperanza	2D	Roble	49000.00	15	28
17	12	Mi Abuela la Hechicera	2D	Ceiba	48000.00	16	29
18	30	Galaxia Nómada	3D	Sauce	44000.00	14	30
19	25	El Jardín de Cristal	2D	Guayacán	36000.00	17	31
20	18	Duelo en la Frontera	2D	Roble	36000.00	17	32
21	22	Océano Profundo	3D	Cedro	0	1	4

Figura 46. *Ejecución de la consulta 4 (adicional) (captura 1 de 2).*

22	16	Código Aurora	3D	Sauce	0	1	6
23	31	Aventuras en el Bosque Encantado	3D	Sauce	0	1	11
24	32	Código Aurora	3D	Sauce	0	1	12
25	26	Código Aurora	3D	Guayacán	0	1	3
26	23	Dragones del Alba	3D	Cedro	0	1	1
27	20	Papá en Apuros	2D	Roble	0	1	8
28	15	Duelo en la Frontera	2D	Ceiba	0	1	10
29	29	Cenizas de Julio	2D	Ceiba	0	1	5
30	27	El Silencio del Valle	2D	Ceiba	0	1	2
31	19	La Ruta del Colibrí	2D	Roble	0	1	9
32	28	Mi Abuela la Hechicera	2D	Ceiba	0	1	7

Figura 47. *Ejecución de la consulta 4 (adicional) (captura 2 de 2).*

Tablas implicadas: funciones, salas, ventas

Propósito de negocio: Esta consulta adicional implementa analítica avanzada de datos al calcular el recaudo total por función y ordenarlas jerárquicamente. Utiliza RANK() particionado por el formato de la película (2D, 3D, etc.) para conocer el podio dentro de cada categoría tecnológica, y ROW_NUMBER() para establecer un ranking general absoluto de taquilla. Esto permite premiar o evaluar el rendimiento comercial de la programación cinematográfica.